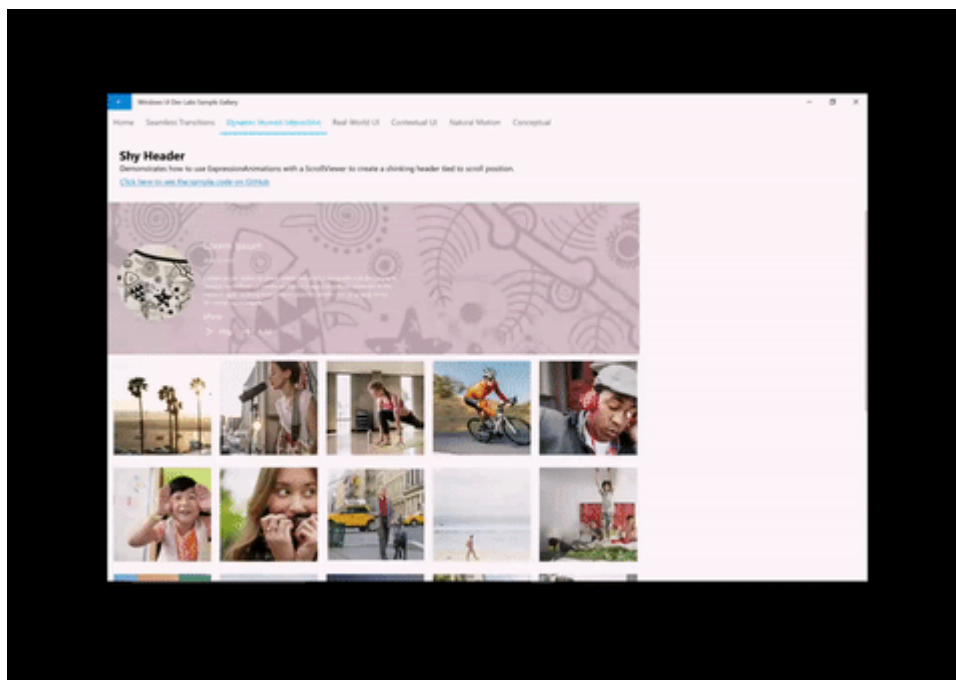
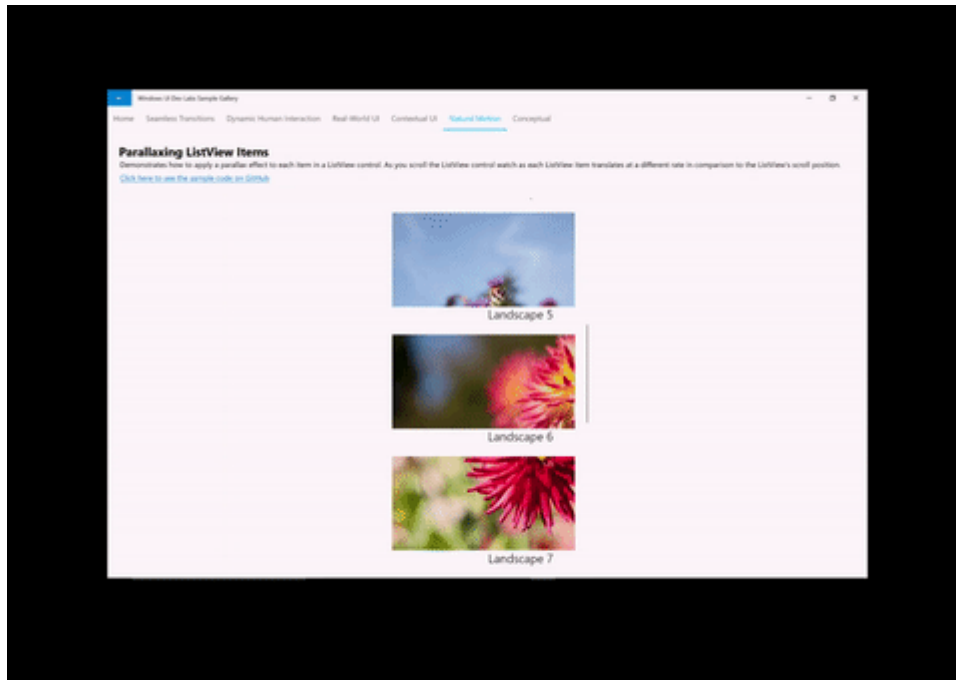


In general, by referencing the position of a ScrollViewer with an ExpressionAnimation, you are able to create an animation that dynamically changes relative the scroll amount.



Using ScrollViewerManipulationPropertySet

To create these dynamic experiences using a XAML ScrollViewer, you must be able to reference the scroll position in an animation. This is done by accessing a CompositionPropertySet off of the XAML ScrollViewer called the ScrollViewerManipulationPropertySet. The ScrollViewerManipulationPropertySet contains a single Vector3 property called Translation that provides access to the scroll position of the ScrollViewer. You can then reference this like any other CompositionPropertySet in your ExpressionAnimation.

General Steps to getting started:

1. Access the `ScrollViewerManipulationPropertySet` via `ElementCompositionPreview`.

- `ElementCompositionPreview.GetScrollViewerManipulationPropertySet(ScrollViewer scroller)`

2. Create an `ExpressionAnimation` that references the `Translation` property from the `PropertySet`.

- Don't forget to set the `Reference Parameter`!

3. Target a `CompositionObject`'s property with the `ExpressionAnimation`.

ⓘ Note

It is recommended that you assign the `PropertySet` returned from the `GetScrollViewerManipulationPropertySet` method to a class variable. This ensures that the property set does not get cleaned up by Garbage Collection and thus does not have any effect on the `ExpressionAnimation` it is referenced in.

`ExpressionAnimations` do not maintain a strong reference to any of the objects used in the equation.

Example

Let's take a look at how the Parallax sample shown above is put together. For reference, all the source code for the app is found in the [Window UI Dev Labs repo on GitHub](#).

The first thing is to get a reference to the `ScrollViewerManipulationPropertySet`.

C#

```
_scrollProperties =  
  
ElementCompositionPreview.GetScrollViewerManipulationPropertySet(myScrollViewer);
```

The next step is to create the `ExpressionAnimation` that defines an equation that utilizes the scroll Position of the `ScrollViewer`.

C#

```
_parallaxExpression = compositor.CreateExpressionAnimation();  
_parallaxExpression.SetScalarParameter("StartOffset", 0.0f);
```

```
_parallaxExpression.SetScalarParameter("ParallaxValue", 0.5f);
_parallaxExpression.SetScalarParameter("ItemHeight", 0.0f);
_parallaxExpression.SetReferenceParameter("ScrollManipulation",
_scrollProperties);
_parallaxExpression.Expression = "(ScrollManipulation.Translation.Y +
StartOffset - (0.5 * ItemHeight)) * ParallaxValue -
(ScrollManipulation.Translation.Y + StartOffset - (0.5 * ItemHeight))";
```

❗ Note

You can also utilize ExpressionBuilder helper classes to construct this same Expression without the need for Strings:

C#

```
var scrollPropSet =
_scrollProperties.GetSpecializedReference<ManipulationPropertySetReferen
ceNode>();
var parallaxValue = 0.5f;
var parallax = (scrollPropSet.Translation.Y + startOffset);
_parallaxExpression = parallax * parallaxValue - parallax;
```

Finally, you take this ExpressionAnimation and target the Visual that you want to parallax. In this case, it is the image for each of the items in the list.

C#

```
Visual visual = ElementCompositionPreview.GetElementVisual(image);
visual.StartAnimation("Offset.Y", _parallaxExpression);
```

Pointer-based animations

Article • 10/20/2022

This article shows how to use the position of a pointer to create dynamic "stick to the cursor" experiences.

Prerequisites

Here, we assume that you're familiar with the concepts discussed in these articles:

- [Input-driven animations](#)
- [Relation based animations](#)

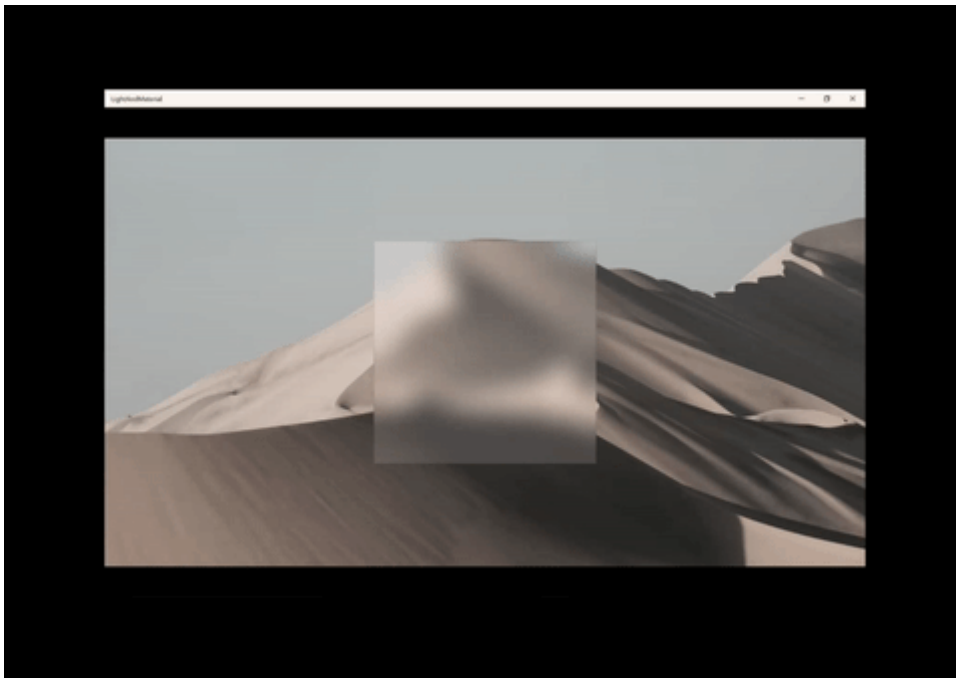
Why Create Pointer Position-Driven Experiences?

In the [Fluent design language](#), touch is not the only way to interact with UI. Because UWP spans across multiple device form factors, end users interact with apps with other input modalities such as Mouse and Pen. Using position data from these other input modalities provides an opportunity to make end users feel even more connected with your app.

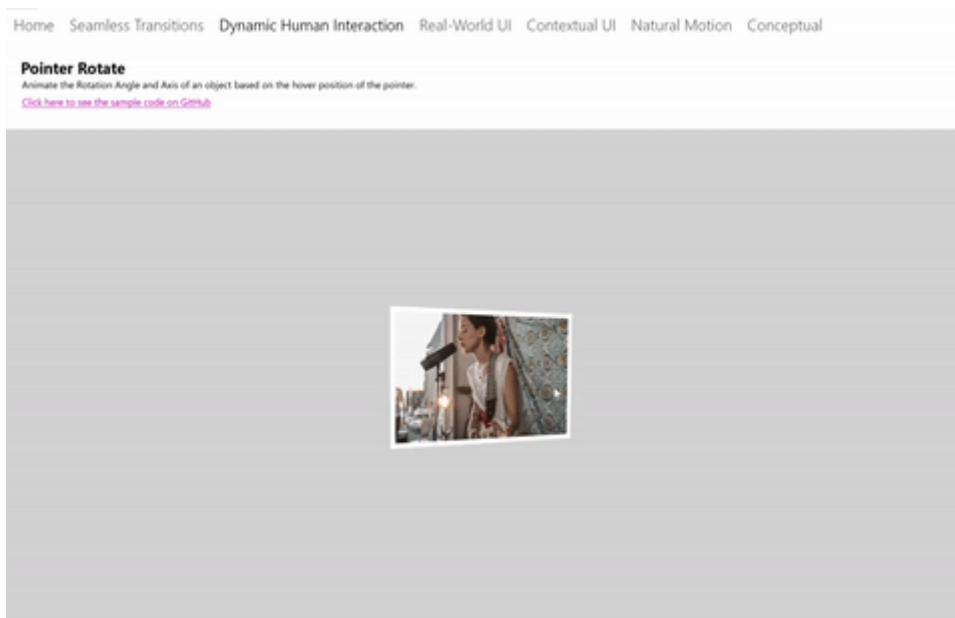
Pointer position-driven experiences let you leverage the on-screen position of a Pointer input modality to create additional motion and UI experiences for your app. These experiences often can provide additional context and feedback to end users about the behavior and structure of the UI. The experience is no longer a one-way stream, but rather starts to become a two-way stream where the end user provides input with their input modality and the app UI can respond back.

Some examples include:

- Animating the position of a [Spotlight](#) to follow the cursor



- Rotating an image based on the position of a pointer



Using PointerPositionPropertySet

You can create these experiences by using the `PointerPositionPropertySet`. This `PropertySet` gets created for a `UIElement` to maintain the position of the pointer while the `UIElement` is positively hit-tested. The position value is relative to the coordinate space of the `UIElement` (a position of $\langle 0,0 \rangle$ is the top left corner of the `UIElement`). You can then leverage this property set in an `Animation` to drive the motion of another property.

For each of the different Pointer Input Modalities, there are a number of input states the input could be in where the position changes: Hover, Pressed, Pressed & Moved. The

PointerPositionPropertySet only maintains the position of the pointer in the Hover, Pressed and Pressed and Moved states for Mouse and Pen.

General steps to get started:

1. Identify the UIElement, you wish to have the position of the pointer tracked in.
2. Access the PointerPositionPropertySet via ElementCompositionPreview.
 - Pass UIElement into the `ElementCompositionPreview.GetPointerPositionPropertySet` method.
3. Create an ExpressionAnimation that references the Position property in the PropertySet.
 - Don't forget to set your reference parameter!
4. Target a CompositionObject's property with the ExpressionAnimation.

ⓘ Note

It is recommended that you assign the PropertySet returned from the `GetPointerPositionPropertySet` method to a class variable. This ensures that the property set does not get cleaned up by Garbage Collection and thus does not have any effect on the ExpressionAnimation it is referenced in.

ExpressionAnimations do not maintain a strong reference to any of the objects used in the equation.

Example

Let's take a look at an example where we leverage the Hover position of a Mouse and Pen input modality to dynamically rotate an image.

Pointer Rotate

Animate the Rotation Angle and Axis of an object based on the hover position of the pointer.

[Click here to see the sample code on GitHub](#)



The image is a `UIElement`, so let's first get a reference to the `PointerPositionPropertySet`

C#

```
_pointerPositionPropSet =  
ElementCompositionPreview.GetPointerPositionPropertySet(UIElement element);
```

In this sample, you have two Expressions at play:

- An Expression where the image rotates based on how far the pointer is from the center of the image. The further away, the more rotation.
- An Expression where the rotation axis changes based on the position of the pointer. You want the rotation axis to be perpendicular to the vector of the position.

Here, you define the two Expressions, one that targets the `RotationAngle` property and the other the `RotationAxis` property. You reference the `PointerPositionPropertySet` like any other `PropertySet`.

In this example, you are building Expressions using the [ExpressionBuilder](#) classes.

C#

```
// using EF = ExpressionBuilder.ExpressionFunctions;  
// || DEFINE THE EXPRESSION FOR THE ROTATION ANGLE ||  
var hoverPosition = _pointerPositionPropSet.GetSpecializedReference  
<PointerPositionPropertySetReferenceNode>().Position;  
var angleExpressionNode =  
EF.Conditional(  
    hoverPosition == new Vector3(0, 0, 0),  
    ExpressionValues.CurrentValue.CreateScalarCurrentValue(),  
    35 * ((EF.Clamp(EF.Distance(center, hoverPosition), 0, distanceToCenter) %
```

```
distanceToCenter) / distanceToCenter));  
_tiltVisual.StartAnimation("RotationAngleInDegrees", angleExpressionNode);  
  
// || DEFINE THE EXPRESSION FOR THE ROTATION AXIS ||  
var axisAngleExpressionNode = EF.Vector3(  
  -(hoverPosition.Y - center.Y) * EF.Conditional(hoverPosition.Y == center.Y,  
    0, 1),  
  (hoverPosition.X - center.X) * EF.Conditional(hoverPosition.X == center.X,  
    0, 1),  
  0);  
_tiltVisual.StartAnimation("RotationAxis", axisAngleExpressionNode);
```

See also

- [Pointer Rotate sample](#) [↗](#)
- [PointerPositionPropertySetReferenceNode](#) class

Custom manipulation experiences with InteractionTracker

Article • 10/20/2022

In this article, we show how to use InteractionTracker to create custom manipulation experiences.

Prerequisites

Here, we assume that you're familiar with the concepts discussed in these articles:

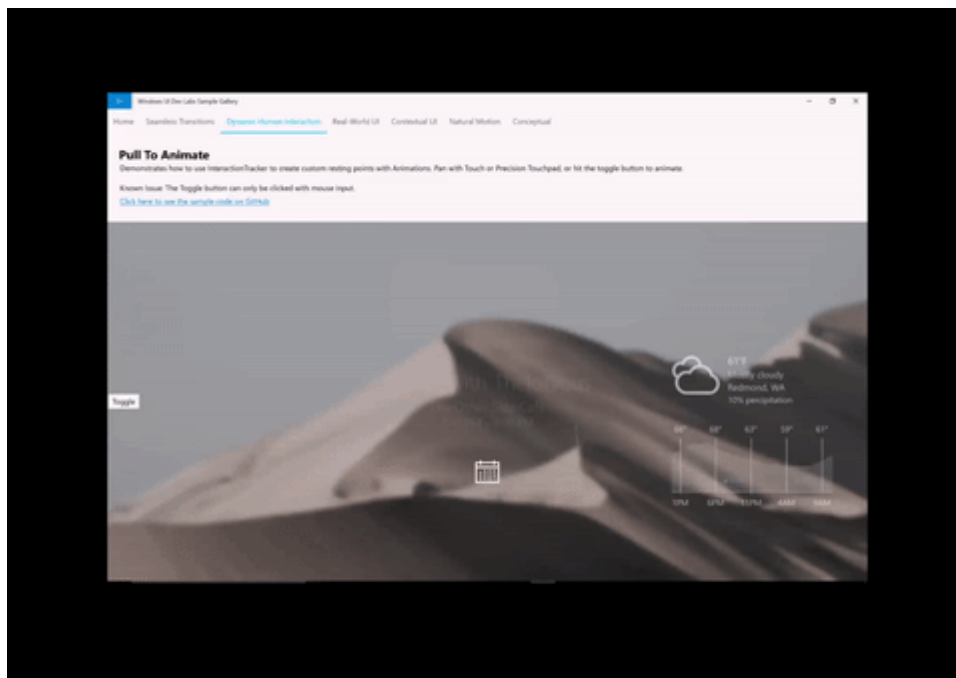
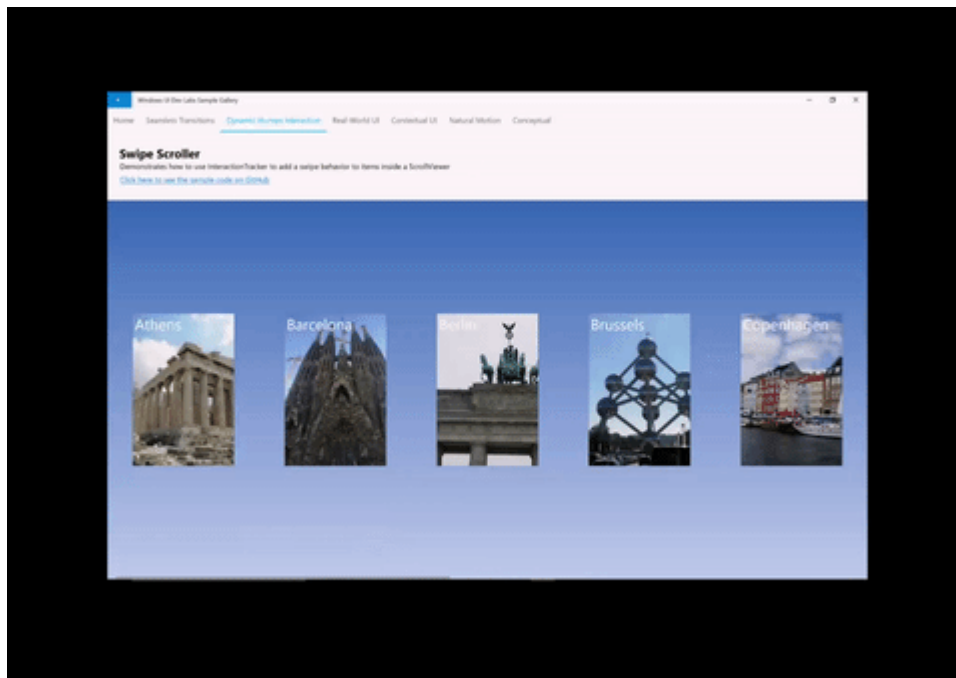
- [Input-driven animations](#)
- [Relation based animations](#)

Why create custom manipulation experiences?

In most cases, utilizing the pre-built manipulation controls are good enough to create UI experiences. But what if you wanted to differentiate from the common controls? What if you wanted to create a specific experience driven by input or have a UI where a traditional manipulation motion is not sufficient? This is where creating custom experiences comes in. They enable app developers and designers to be more creative – bring to life motion experiences that better exemplify their branding and custom design language. From the ground up, you are given access to the right set of building blocks to completely customize a manipulation experience – from how motion should respond with the finger on and off the screen to snap points and input chaining.

Below are some common examples of when you'd create a custom manipulation experience:

- Adding a custom swipe, delete/dismiss behavior
- Input driven effects (panning causes content to blur)
- Custom Controls with tailored manipulation motions (custom ListView, ScrollView, etc.)



Why use InteractionTracker?

InteractionTracker was introduced to the Windows.UI.Composition.Interactions namespace in the 10586 SDK version. InteractionTracker enables:

- **Complete Flexibility** – we want you to be able to customize and tailor every aspect of a manipulation experience; specifically, the exact motions that occur during, or in response to, input. When building a custom manipulation experience with InteractionTracker, all the knobs you need are at your disposal.
- **Smooth Performance** – one of the challenges with manipulation experiences is that their performance is dependent on the UI thread. This can negatively impact any manipulation experience when the UI is busy. InteractionTracker was built to

utilize the new Animation engine that operates on an independent thread at 60 FPS, resulting in smooth motion.

Overview: InteractionTracker

When creating custom manipulation experiences, there are two primary components you interact with. We'll discuss these first:

- [InteractionTracker](#) – the core object maintaining a state machine whose properties are driven by active user input or direct updates and animations. It is intended to then tie to a [CompositionAnimation](#) to create the custom manipulation motion.
- [VisualInteractionSource](#) – a complement object that defines when and under what conditions input gets sent to [InteractionTracker](#). It defines both the [CompositionVisual](#) used for Hit-testing as well as other input configuration properties.

As a state machine, properties of [InteractionTracker](#) can be driven by any of the following:

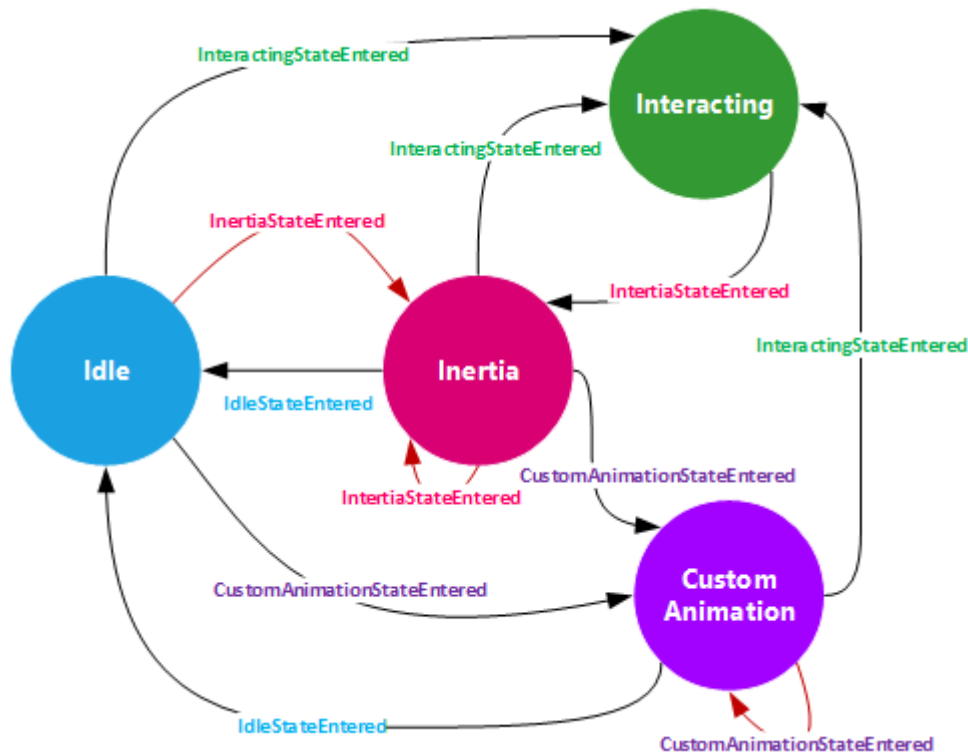
- Direct User Interaction – the end user is directly manipulating within the [VisualInteractionSource](#) hit-test region
- Inertia – either from programmatic velocity or a user gesture, properties of [InteractionTracker](#) animate under an inertia curve
- CustomAnimation – a custom animation directly targeting a property of [InteractionTracker](#)

InteractionTracker State Machine

As mentioned previously, [InteractionTracker](#) is a state machine with 4 states – each of which can transition to any of the other four states. (For more info about how [InteractionTracker](#) transitions between these states, see the [InteractionTracker](#) class documentation.)

State	Description
Idle	No active, driving inputs or animations
Interacting	Active user input detected
Inertia	Active motion resulting from active input or programmatic velocity
CustomAnimation	Active motion resulting from a custom animation

In each of the cases where the state of InteractionTracker changes, an event (or callback) is generated that you can listen for. In order for you to listen for these events, they must implement the [InteractionTrackerOwner](#) interface and create their InteractionTracker object with the CreateWithOwner method. The following diagram also outlines when the different events get triggered.



Using the VisualInteractionSource

For InteractionTracker to be driven by Input, you need to connect a VisualInteractionSource (VIS) to it. The VIS is created as a complement object using a CompositionVisual to define:

1. The hit-test region that input will be tracked, and the coordinate space gestures are detected in
2. The configurations of input that will get detected and routed, a few include:
 - Detectable gestures: Position X and Y (horizontal and vertical panning), Scale (pinch)
 - Inertia
 - Rails & Chaining
 - Redirection Modes: what input data is redirected automatically to InteractionTracker

Because the VisualInteractionSource is created based off the hit-test position and coordinate space of a Visual, it is recommended not to use a Visual that will be in motion or changing position.

ⓘ Note

You can use multiple VisualInteractionSource instances with the same InteractionTracker if there are multiple hit-test regions. However, the most common case is to use only one VIS.

The VisualInteractionSource is also responsible for managing when input data from different modalities (touch, PTP, Pen) get routed to InteractionTracker. This behavior is defined by the ManipulationRedirectionMode property. By default, all Pointer input is sent to the UI thread and Precision Touchpad input goes to the VisualInteractionSource and InteractionTracker.

Thus, if you want to have Touch and Pen (Creators Update) to drive a manipulation through a VisualInteractionSource and InteractionTracker, you must call the VisualInteractionSource.TryRedirectForManipulation method. In the short snippet below from a XAML app, the method is called when a touch pressed event occurs at the top most UIElement Grid:

C#

```
private void root_PointerPressed(object sender, PointerRoutedEventArgs e)
{
    if (e.Pointer.PointerDeviceType ==
        Windows.Devices.Input.PointerDeviceType.Touch)
    {
        _source.TryRedirectForManipulation(e.GetCurrentPoint(root));
    }
}
```

Tie-in with ExpressionAnimations

When utilizing InteractionTracker to drive a manipulation experience, you interact primarily with the Scale and Position properties. Like other CompositionObject properties, these properties can be both the target and referenced in a CompositionAnimation, most commonly ExpressionAnimations.

To use InteractionTracker within an Expression, reference the tracker's Position (or Scale) property like in the example below. As the property of InteractionTracker is modified

due to any of the conditions described earlier, the output of the Expression changes as well.

C#

```
// With Strings
var opacityExp = _compositor.CreateExpressionAnimation("-tracker.Position");
opacityExp.SetReferenceParameter("tracker", _tracker);

// With ExpressionBuilder
var opacityExp = -_tracker.GetReference().Position;
```

ⓘ Note

When referencing InteractionTracker's position in an Expression, you must negate the value for the resulting Expression to move in the correct direction. This is because InteractionTracker's progression from origin on a graph and allow you to think about InteractionTracker's progression in "real world" coordinates, such as distance from its origin.

Get Started

To get started with using InteractionTracker to create custom manipulation experiences:

1. Create your InteractionTracker object using InteractionTracker.Create or InteractionTracker.CreateWithOwner.
 - (If you use CreateWithOwner, ensure you implement the InteractionTrackerOwner interface.)
2. Set the Min and Max position of your newly created InteractionTracker.
3. Create your VisualInteractionSource with a CompositionVisual.
 - Ensure the visual you pass in has a non-zero size. Otherwise, it will not get hit-tested correctly.
4. Set the properties of the VisualInteractionSource.
 - VisualInteractionSourceRedirectionMode
 - PositionXSourceMode, PositionYSourceMode, ScaleSourceMode
 - Rails & Chaining

5. Add the VisualInteractionSource to InteractionTracker using InteractionTracker.InteractionSources.Add.
6. Setup TryRedirectForManipulation for when Touch and Pen input is detected.
 - For XAML, this is typically done on the UIElement's PointerPressed event.
7. Create an ExpressionAnimation that references InteractionTracker's position and target a CompositionObject's property.

Here is a short code snippet that shows #1 – 5 in action:

C#

```
private void InteractionTrackerSetup(Compositor compositor, Visual hitTestRoot)
{
    // #1 Create InteractionTracker object
    var tracker = InteractionTracker.Create(compositor);

    // #2 Set Min and Max positions
    tracker.MinPosition = new Vector3(-1000f);
    tracker.MaxPosition = new Vector3(1000f);

    // #3 Setup the VisualInteractionSource
    var source = VisualInteractionSource.Create(hitTestRoot);

    // #4 Set the properties for the VisualInteractionSource
    source.ManipulationRedirectionMode =
        VisualInteractionSourceRedirectionMode.CapableTouchpadOnly;
    source.PositionXSourceMode = InteractionSourceMode.EnabledWithInertia;
    source.PositionYSourceMode = InteractionSourceMode.EnabledWithInertia;

    // #5 Add the VisualInteractionSource to InteractionTracker
    tracker.InteractionSources.Add(source);
}
```

For more advanced usages of InteractionTracker, see the following articles:

- [Create snap points with InertiaModifiers](#)
- [Pull-to-refresh with SourceModifiers](#)

Create snap points with inertia modifiers

Article • 10/20/2022

In this article, we take a deeper dive into how to use an InteractionTracker's InertiaModifier feature to create motion experiences that snap to a specified point.

Prerequisites

Here, we assume that you're familiar with the concepts discussed in these articles:

- [Input-driven animations](#)
- [Custom manipulation experiences with InteractionTracker](#)
- [Relation based animations](#)

What are snap points and why are they useful?

When building custom manipulation experiences, sometimes it is helpful to create specialized *position points* within the scrollable/zoomable canvas that InteractionTracker will always come to rest at. These are often called *snap points*.

Notice in the following example how the scrolling can leave the UI in an awkward position between the different images:



If you add snap points, when you stop scrolling between the images, they "snap" to a specified position. With snap points, it makes the experience of scrolling through images much cleaner and more responsive.



InteractionTracker and InertiaModifiers

When building customized manipulation experiences with InteractionTracker, you can create snap point motion experiences by utilizing InertiaModifiers. InertiaModifiers are essentially a way for you to define where or how InteractionTracker reaches its destination when entering the Inertia state. You can apply InertiaModifiers to impact the X or Y position or Scale properties of InteractionTracker.

There are 3 types of InertiaModifiers:

- InteractionTrackerInertiaRestingValue – a way to modify the final resting position after an interaction or programmatic velocity. A predefined motion will take InteractionTracker to that position.
- InteractionTrackerInertiaMotion – a way to define a specific motion InteractionTracker will perform after an interaction or programmatic velocity. The final position will be derived from this motion.
- InteractionTrackerInertiaNaturalMotion – a way to define the final resting position after an interaction or programmatic velocity but with a physics based animation (NaturalMotionAnimation).

When entering Inertia, InteractionTracker evaluates each of the InertiaModifiers assigned to it and determines if any of them apply. This means you can create and assign multiple

InertiaModifiers to an InteractionTracker, But, when defining each, you need to do the following:

1. Define the Condition – an Expression that defines the conditional statement when this specific InertiaModifier should take place. This often requires looking at InteractionTracker's NaturalRestingPosition (destination given default Inertia).
2. Define the RestingValue/Motion/NaturalMotion – define the actual Resting Value Expression, Motion Expression or NaturalMotionAnimation that takes place when the condition is met.

⚠ Note

The condition aspect of the InertiaModifiers are only evaluated once when InteractionTracker enters Inertia. However, only for InertiaMotion, the motion Expression is evaluated every frame for the modifier whose condition is true.

Example

Now let's look at how you can use InertiaModifiers to create some snap point experiences to recreate the scrolling canvas of images. In this example, each manipulation is meant to potentially move through a single image – this is often referred to as Single Mandatory Snap Points.

Let's start by setting up InteractionTracker, the VisualInteractionSource and the Expression that will leverage the position of InteractionTracker.

C#

```
private void SetupInput()
{
    _tracker = InteractionTracker.Create(_compositor);
    _tracker.MinPosition = new Vector3(0f);
    _tracker.MaxPosition = new Vector3(3000f);

    _source = VisualInteractionSource.Create(_root);
    _source.ManipulationRedirectionMode =
        VisualInteractionSourceRedirectionMode.CapableTouchpadOnly;
    _source.PositionYSourceMode = InteractionSourceMode.EnabledWithInertia;
    _tracker.InteractionSources.Add(_source);

    var scrollExp = _compositor.CreateExpressionAnimation("-
tracker.Position.Y");
    scrollExp.SetReferenceParameter("tracker", _tracker);

    ElementCompositionPreview.GetElementVisual(scrollPanel).StartAnimation("Offs
```

```
et.Y", scrollExp);  
}
```

Next, because a Single Mandatory Snap Point behavior either will move the content up or down, you will need two different inertia modifiers: one that moves the Scrollable content up, and one that moves it down.

C#

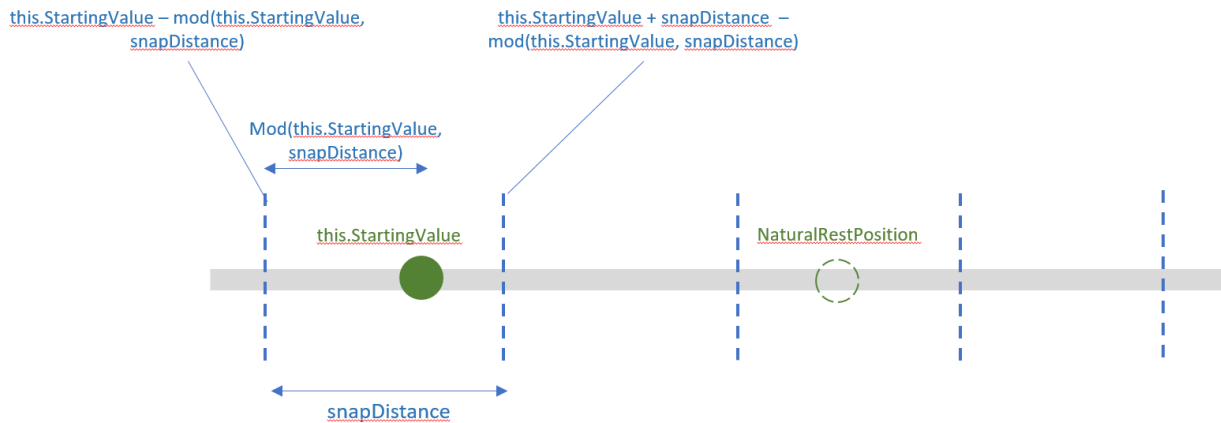
```
// Snap-Point to move the content up  
var snapUpModifier =  
InteractionTrackerInertiaRestingValue.Create(_compositor);  
// Snap-Point to move the content down  
var snapDownModifier =  
InteractionTrackerInertiaRestingValue.Create(_compositor);
```

Whether to snap up or down is determined based on where InteractionTracker naturally would land within relative to the snap distance – the distance between the snap locations. If past the halfway point, then snap down, otherwise snap up. (In this example, you store the snap distance in a PropertySet)

C#

```
// Is NaturalRestingPosition less than the halfway point between Snap  
Points?  
snapUpModifier.Condition = _compositor.CreateExpressionAnimation(  
"this.Target.NaturalRestingPosition.y < (this.StartingValue - " +  
"mod(this.StartingValue, prop.snapDistance) + prop.snapDistance / 2)");  
snapUpModifier.Condition.SetReferenceParameter("prop", _propSet);  
// Is NaturalRestingPosition greater than the halfway point between Snap  
Points?  
snapDownModifier.Condition = _compositor.CreateExpressionAnimation(  
"this.Target.NaturalRestingPosition.y >= (this.StartingValue - " +  
"mod(this.StartingValue, prop.snapDistance) + prop.snapDistance / 2)");  
snapDownModifier.Condition.SetReferenceParameter("prop", _propSet);
```

This diagram gives a visual description to the logic that is happening:



Now you just need to define the Resting Values for each InertiaModifier: either move the position of InteractionTracker to the previous snap position or the next one.

C#

```

snapUpModifier.RestingValue = _compositor.CreateExpressionAnimation(
    "this.StartingValue - mod(this.StartingValue, prop.snapDistance)");
snapUpModifier.RestingValue.SetReferenceParameter("prop", _propSet);
snapDownModifier.RestingValue = _compositor.CreateExpressionAnimation(
    "this.StartingValue + prop.snapDistance - mod(this.StartingValue, " +
    "prop.snapDistance)");
snapDownModifier.RestingValue.SetReferenceParameter("prop", _propSet);

```

Finally, add the InertiaModifiers to InteractionTracker. Now when InteractionTracker enters its InertiaState, it will check the conditions of your InertiaModifiers to see if its position should be modified.

C#

```

var modifiers = new InteractionTrackerInertiaRestingValue[] {
    snapUpModifier, snapDownModifier };
_tracker.ConfigurePositionYInertiaModifiers(modifiers);

```

Pull-to-refresh with source modifiers

Article • 10/20/2022

In this article, we take a deeper dive into how to use an InteractionTracker's SourceModifier feature and demonstrate its use by creating a custom pull-to-refresh control.

Prerequisites

Here, we assume that you're familiar with the concepts discussed in these articles:

- [Input-driven animations](#)
- [Custom manipulation experiences with InteractionTracker](#)
- [Relation based animations](#)

What is a SourceModifier and why are they useful?

Like [InertiaModifiers](#), SourceModifiers give you finer grain control over the motion of an InteractionTracker. But unlike InertiaModifiers that define the motion after InteractionTracker enters inertia, SourceModifiers define the motion while InteractionTracker is still in its interacting state. In these cases, you want a different experience than the traditional "stick to the finger".

A classic example of this is the pull-to-refresh experience - when the user pulls the list to refresh the contents and the list pans at the same speed as the finger and stops after a certain distance, the motion would feel abrupt and mechanical. A more natural experience would be to introduce a feel of resistance while the user actively interacts with the list. This small nuance helps make the overall end user experience of interacting with a list more dynamic and appealing. In the Example section, we go into more detail about how to build this.

There are 2 types of Source Modifiers:

- **DeltaPosition** – is the delta between the current frame position and the previous frame position of the finger during the touch pan interaction. This source modifier allows you to modify the delta position of the interaction before sending it for further processing. This is a Vector3 type parameter and the developer can choose to modify any of the X or Y or Z attributes of the position before passing it to the InteractionTracker.

- DeltaScale - is the delta between the current frame scale and the previous frame scale that was applied during the touch zoom interaction. This source modifier allows you to modify the zooming level of the interaction. This is a float type attribute that the developer can modify before passing it to the InteractionTracker.

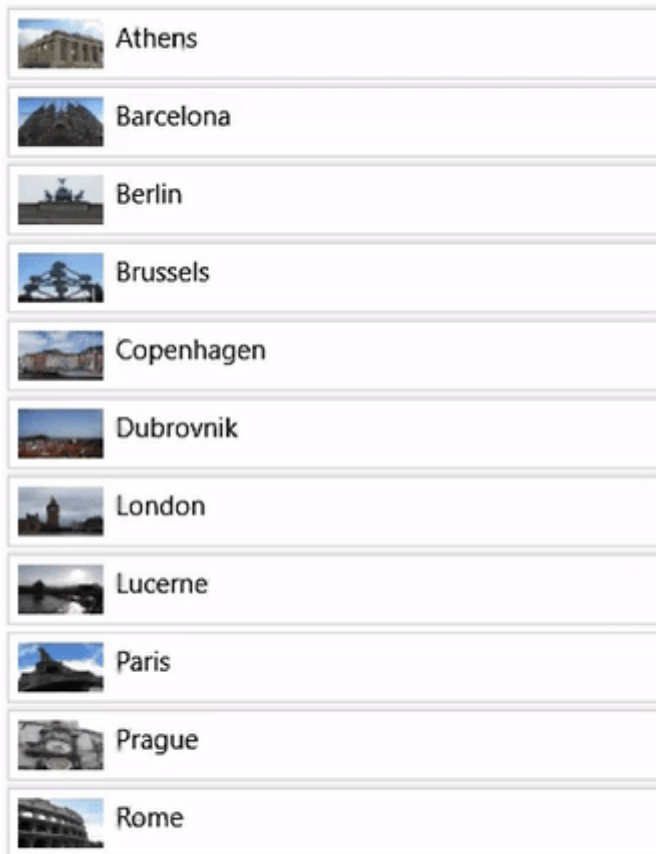
When InteractionTracker is in its Interacting state, it evaluates each of the Source Modifiers assigned to it and determines if any of them apply. This means you can create and assign multiple Source Modifiers to an InteractionTracker. But, when defining each, you need to do the following:

1. Define the Condition – an Expression that defines the conditional statement when this specific Source Modifier should be applied.
2. Define the DeltaPosition/DeltaScale – The source modifier expression that alters the DeltaPosition or DeltaScale when the above defined condition is met.

Example

Now let's look at how you can use Source Modifiers to create a custom pull-to-refresh experience with an existing XAML ListView Control. We will be using a Canvas as the "Refresh Panel" that will be stacked on top of a XAML ListView to build this experience.

For the end user experience, we want to create the effect of "resistance" as the user is actively panning the list (with touch) and stop panning after the position goes beyond a certain point.



The working code for this experience can be found in the [Window UI Dev Labs repo on GitHub](#). Here is the step-by-step walk through of building that experience. In your XAML markup code, you have the following:

XAML

```
<StackPanel Height="500" MaxHeight="500" x:Name="ContentPanel"
HorizontalAlignment="Left" VerticalAlignment="Top" >
    <Canvas Width="400" Height="100" x:Name="RefreshPanel" >
        <Image x:Name="FirstGear" Source="ms-appx:///Assets/Loading.png" Width="20"
Height="20" Canvas.Left="200" Canvas.Top="70"/>
    </Canvas>
    <ListView x:Name="ThumbnailList"
MaxWidth="400"
Height="500"
ScrollViewer.VerticalScrollMode="Enabled"
ScrollViewer.IsScrollInertiaEnabled="False"
ScrollViewer.IsVerticalScrollChainingEnabled="True" >
        <ListView.ItemTemplate>
            .....
        </ListView.ItemTemplate>
    </ListView>
</StackPanel>
```

Because the ListView (`ThumbnailList`) is a XAML control that already scrolls, you need the scrolling to be chained up to its parent (`ContentPanel`) when it reaches the topmost item and can't scroll anymore. (`ContentPanel` is where you will apply the Source Modifiers.) For this to happen you need to set `ScrollViewer.IsVerticalScrollChainingEnabled` to **true** in the ListView markup. You will also need to set the chaining mode on the `VisualInteractionSource` to **Always**.

You need to set `PointerPressedEvent` handler with the *handledEventsToo* parameter as **true**. Without this option, the `PointerPressedEvent` will not be chained to the `ContentPanel` as the `ListView` control will mark those events as handled and they will not be sent up the visual chain.

C#

```
//The PointerPressed handler needs to be added using AddHandler method with
the //handledEventsToo boolean set to "true"
//instead of the XAML element's "PointerPressed=Window_PointerPressed",
//because the list view needs to chain PointerPressed handled events as
well.
ContentPanel.AddHandler(PointerPressedEvent, new PointerEventHandler(
Window_PointerPressed), true);
```

Now, you're ready to tie this with `InteractionTracker`. Start by setting up `InteractionTracker`, the `VisualInteractionSource` and the Expression that will leverage the position of `InteractionTracker`.

C#

```
// InteractionTracker and VisualInteractionSource setup.
_root = ElementCompositionPreview.GetElementVisual(Root);
_compositor = _root.Compositor;
_tracker = InteractionTracker.Create(_compositor);
_interactionSource = VisualInteractionSource.Create(_root);
_interactionSource.PositionYSourceMode =
InteractionSourceMode.EnabledWithInertia;
_interactionSource.PositionYChainingMode = InteractionChainingMode.Always;
_tracker.InteractionSources.Add(_interactionSource);
float refreshPanelHeight = (float)RefreshPanel.ActualHeight;
_tracker.MaxPosition = new Vector3((float)Root.ActualWidth, 0, 0);
_tracker.MinPosition = new Vector3(-(float)Root.ActualWidth, -
refreshPanelHeight, 0);

// Use the Tacker's Position (negated) to apply to the Offset of the Image.
// The -{refreshPanelHeight} is to hide the refresh panel
m_positionExpression = _compositor.CreateExpressionAnimation($"-
tracker.Position.Y - {refreshPanelHeight} ");
m_positionExpression.SetReferenceParameter("tracker", _tracker);
_contentPanelVisual.StartAnimation("Offset.Y", m_positionExpression);
```


With this set up, the refresh panel is out of the viewport in its starting position and all the user sees is the listView. When the panning reaches the ContentPanel, the PointerPressed event will be fired, where you ask the System to use InteractionTracker to drive the manipulation experience.

C#

```
private void Window_PointerPressed(object sender, PointerRoutedEventArgs e)
{
    if (e.Pointer.PointerDeviceType ==
        Windows.Devices.Input.PointerDeviceType.Touch) {
        // Tell the system to use the gestures from this pointer point (if it can).
        _interactionSource.TryRedirectForManipulation(e.GetCurrentPoint(null));
    }
}
```

ⓘ Note

If chaining Handled events is not needed, adding PointerPressedEvent handler can be done directly through XAML markup using the attribute (PointerPressed="Window_PointerPressed").

The next step is to set up the source modifiers. You will be using 2 source modifiers to get this behavior; *Resistance* and *Stop*.

- Resistance – Move the DeltaPosition.Y at half the speed until it reaches the height of the RefreshPanel.

C#

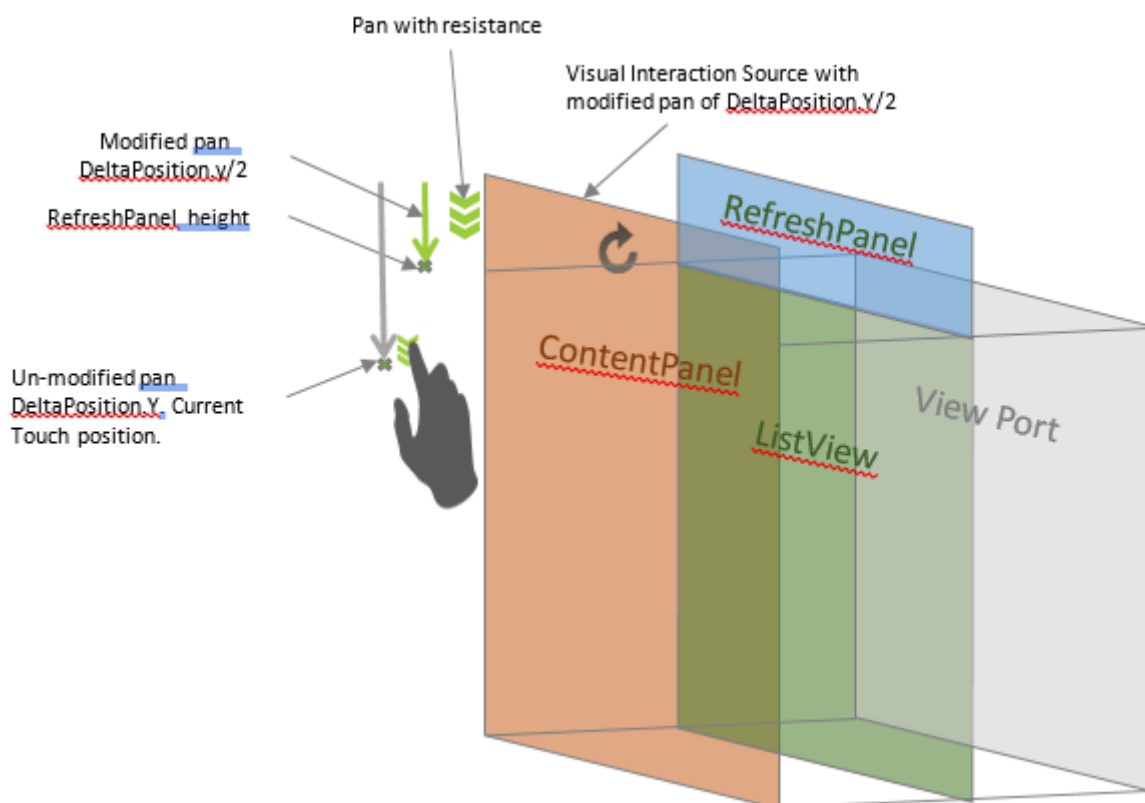
```
CompositionConditionalValue resistanceModifier =
    CompositionConditionalValue.Create (_compositor);
ExpressionAnimation resistanceCondition =
    _compositor.CreateExpressionAnimation(
        $"-tracker.Position.Y < {pullToRefreshDistance}");
resistanceCondition.SetReferenceParameter("tracker", _tracker);
ExpressionAnimation resistanceAlternateValue =
    _compositor.CreateExpressionAnimation(
        "source.DeltaPosition.Y / 3");
resistanceAlternateValue.SetReferenceParameter("source",
    _interactionSource);
resistanceModifier.Condition = resistanceCondition;
resistanceModifier.Value = resistanceAlternateValue;
```

- Stop – Stop moving after the entire RefreshPanel is on the screen.

C#

```
CompositionConditionalValue stoppingModifier =  
CompositionConditionalValue.Create (_compositor);  
ExpressionAnimation stoppingCondition =  
_compositor.CreateExpressionAnimation(  
    $"-tracker.Position.Y >= {pullToRefreshDistance}");  
stoppingCondition.SetReferenceParameter("tracker", _tracker);  
ExpressionAnimation stoppingAlternateValue =  
_compositor.CreateExpressionAnimation("0");  
stoppingModifier.Condition = stoppingCondition;  
stoppingModifier.Value = stoppingAlternateValue;  
Now add the 2 source modifiers to the InteractionTracker.  
List<CompositionConditionalValue> modifierList = new  
List<CompositionConditionalValue>()  
{ resistanceModifier, stoppingModifier };  
_interactionSource.ConfigureDeltaPositionYModifiers(modifierList);
```

This diagram gives a visualization of the SourceModifiers setup.



Now with the SourceModifiers, you will notice when panning the ListView down and reach the topmost item, the refresh panel is pulled down at half the pace of the pan until it reaches the RefreshPanel height and then stops moving.

In the full sample, a keyframe animation is used to spin an icon during interaction in the RefreshPanel canvas. Any content can be used in its place or utilize InteractionTracker's position to drive that animation separately.

Composition effects

Article • 03/16/2023

The [Windows.UI.Composition](#) APIs allows real-time effects to be applied to images and UI with animatable effect properties. In this overview, we'll run through the functionality available that allows effects to be applied to a composition visual.

To support [Universal Windows Platform \(UWP\)](#) consistency for developers describing effects in their applications, composition effects leverage Win2D's `IGraphicsEffect` interface to use effect descriptions via the [Microsoft.Graphics.Canvas.Effects](#) [Namespace](#).

Brush effects are used to paint areas of an application by applying effects to a set of existing images. Windows 10 composition effect APIs are focused on Sprite Visuals. The `SpriteVisual` allows for flexibility and interplay in color, image and effect creation. The `SpriteVisual` is a composition visual type that can fill a 2D rectangle with a brush. The visual defines the bounds of the rectangle and the brush defines the pixels used to paint the rectangle.

Effect brushes are used on composition tree visuals whose content comes from the output of an effect graph. Effects can reference existing surfaces/textures, but not the output of other composition trees.

Effects can also be applied to XAML UIElements using an effect brush with [XamlCompositionBrushBase](#).

Effect Features

- [Effect Library](#)
- [Chaining Effects](#)
- [Animation Support](#)
- [Constant vs. Animated Effect Properties](#)
- [Multiple Effect Instances with Independent Properties](#)

Effect Library

Currently composition supports the following effects:

Effect	Description
--------	-------------

Effect	Description
2D affine transform	Applies a 2D affine transform matrix to an image.
Arithmetic composite	Combines two images using a flexible equation.
Blend effect	Creates a blend effect that combines two images. Composition provides 21 of the 26 blend modes supported in Win2D.
Color source	Generates an image containing a solid color.
Composite	Combines two images. Composition provides all 13 composite modes supported in Win2D.
Contrast	Increases or decreases the contrast of an image.
Exposure	Increases or decreases the exposure of an image.
Grayscale	Converts an image to monochromatic gray.
Gamma transfer	Alters the colors of an image by applying a per-channel gamma transfer function.
Hue rotate	Alters the color of an image by rotating its hue values.
Invert	Inverts the colors of an image.
Saturate	Alters the saturation of an image.
Sepia	Converts an image to sepia tones.
Temperature and tint	Adjusts the temperature and/or tint of an image.

See Win2D's [Microsoft.Graphics.Canvas.Effects](#) Namespace for more detailed information. Effects not supported in composition are noted as [NoComposition].

Chaining Effects

Effects can be chained, allowing an application to simultaneously use multiple effects on an image. Effect graphs can support multiple effects that can refer to one and other. When describing your effect, simply add an effect as input to your effect.

C#

```
IGraphicsEffect graphicsEffect =
    new Microsoft.Graphics.Canvas.Effects.ArithmeticCompositeEffect
    {
```

```

Source1 = new CompositionEffectSourceParameter("source1"),
Source2 = new SaturationEffect
{
    Saturation = 0,
    Source = new CompositionEffectSourceParameter("source2")
},
MultiplyAmount = 0,
Source1Amount = 0.5f,
Source2Amount = 0.5f,
Offset = 0
}

```

The example above describes an arithmetic composite effect which has two inputs. The second input has a saturation effect with a .5 saturation property.

Animation Support

Effect properties support animation, during effect compilation you can specify effect properties can be animated and which can be "baked in" as constants. The animatable properties are specified through strings of the form "effect name.property name". These properties can be animated independently over multiple instantiations of the effect.

Constant vs Animated Effect Properties

During effect compilation you can specify effect properties as dynamic or as properties that are "baked in" as constants. The dynamic properties are specified through strings of the form "<effect name>.<property name>". The dynamic properties can be set to a specific value or can be animated using the composition animation system.

When compiling the effect description above, you have the flexibility of either baking in saturation to be equal to 0.5 or making it dynamic and setting it dynamically or animating it.

Compiling an effect with saturation baked in:

C#

```
var effectFactory = _compositor.CreateEffectFactory(graphicsEffect);
```

Compiling an effect with dynamic saturation:

C#

```
var effectFactory = _compositor.CreateEffectFactory(graphicsEffect, new[]
{"SaturationEffect.Saturation"});
```

```
_catEffect = effectFactory.CreateBrush();  
_catEffect.SetSourceParameter("mySource", surfaceBrush);  
_catEffect.Properties.InsertScalar("saturationEffect.Saturation", 0f);
```

The saturation property of the effect above can then be either set to a static value or animated using either Expression or ScalarKeyFrame animations.

You can create a ScalarKeyFrame that will be used to animate the Saturation property of an effect like this:

C#

```
ScalarKeyFrameAnimation effectAnimation =  
_compositor.CreateScalarKeyFrameAnimation();  
    effectAnimation.InsertKeyFrame(0f, 0f);  
    effectAnimation.InsertKeyFrame(0.50f, 1f);  
    effectAnimation.InsertKeyFrame(1.0f, 0f);  
    effectAnimation.Duration = TimeSpan.FromMilliseconds(2500);  
    effectAnimation.IterationBehavior =  
AnimationIterationBehavior.Forever;
```

Start the animation on the Saturation property of the effect like this:

C#

```
catEffect.Properties.StartAnimation("saturationEffect.Saturation",  
effectAnimation);
```

Multiple Effect Instances with Independent Properties

By specifying that a parameter should be dynamic during effect compilation, the parameter can then be changed on a per-effect instance basis. This allows two Visuals to use the same effect but be rendered with different effect properties.

Getting Started with Composition Effects

This quick start tutorial shows you how to make use of some of the basic capabilities of effects.

- [Installing Visual Studio](#)
- [Creating a new project](#)
- [Installing Win2D](#)
- [Setting your Composition Basics](#)
- [Creating a CompositionSurface Brush](#)

- [Creating, Compiling and Applying Effects](#)

Installing Visual Studio

- If you don't have a supported version of Visual Studio installed, go to the Visual Studio Downloads page [here](#) [↗].

Creating a new project

- Go to File->New->Project...
- Select 'Visual C#'
- Create a 'Blank App (Windows Universal)' (Visual Studio 2015)
- Enter a project name of your choosing
- Click 'OK'

Installing Win2D

Win2D is released as a Nuget.org package and needs to be installed before you can use effects.

There are two package versions, one for Windows 10 and one for Windows 8.1. For Composition effects you'll use the Windows 10 version.

- Launch the NuGet Package Manager by going to Tools → NuGet Package Manager → Manage NuGet Packages for Solution.
- Search for "Win2D" and select the appropriate package for your target version of Windows. Because Windows.UI.Composition supports Windows 10 (not 8.1), select Win2D.uwp.
- Accept the license agreement
- Click 'Close'

In the next few steps we will use composition API's to apply a saturation effect to this cat image which will remove all saturation. In this model the effect is created and then applied to an image.



Setting your Composition Basics

C#

```
_compositor = new Compositor();  
_root = _compositor.CreateContainerVisual();  
_target = _compositor.CreateTargetForCurrentView();  
_target.Root = _root;  
_imageFactory = new CompositionImageFactory(_compositor)  
Desaturate();
```

Creating a CompositionSurface Brush

C#

```
CompositionSurfaceBrush surfaceBrush = _compositor.CreateSurfaceBrush();  
LoadImage(surfaceBrush);
```

Creating, Compiling and Applying Effects

1. Create the graphics effect

C#

```
var graphicsEffect = new SaturationEffect  
{  
    Saturation = 0.0f,  
    Source = new CompositionEffectSourceParameter("mySource")  
};
```


2. Compile the effect and create effect brush

```
C#  
  
var effectFactory = _compositor.CreateEffectFactory(graphicsEffect);  
  
var catEffect = effectFactory.CreateBrush();  
catEffect.SetSourceParameter("mySource", surfaceBrush);
```

3. Create a SpriteVisual in the composition tree and apply the effect

```
C#  
  
var catVisual = _compositor.CreateSpriteVisual();  
catVisual.Brush = catEffect;  
catVisual.Size = new Vector2(219, 300);  
_root.Children.InsertAtBottom(catVisual);
```

4. Create your image source to load.

```
C#  
  
CompositionImage imageSource = _imageFactory.CreateImageFromUri(new  
Uri("ms-appx:///Assets/cat.png"));  
CompositionImageLoadResult result = await  
imageSource.CompleteLoadAsync();  
if (result.Status == CompositionImageLoadStatus.Success)
```

5. Size and brush the surface on the SpriteVisual

```
C#  
  
brush.Surface = imageSource.Surface;
```

6. Run your app – your results should be a desaturated cat:



More Information

- [Microsoft – Composition GitHub](#) ↗
- **[Windows.UI.Composition](#)**
- [Composition Overview](#) ↗
- [Visual Tree Basics](#)
- [Composition Brushes](#)
- [XamlCompositionBrushBase](#)
- [Animation Overview](#)
- [Composition native DirectX and Direct2D interoperation with BeginDraw and EndDraw](#)

Shadows in Windows UI

Article • 03/16/2023

The [DropShadow](#) class provides means of creating a configurable shadow that can be applied to a [SpriteVisual](#) or [LayerVisual](#) (subtree of Visuals). As is customary for objects in the Visual Layer, all properties of the DropShadow can be animated using CompositionAnimations.

Basic drop shadow

To create a basic shadow, simply create a new DropShadow and associate it to your visual. The shadow is rectangular by default. A standard set of properties are available to tweak the look and feel of your shadow.

C#

```
var basicRectVisual = _compositor.CreateSpriteVisual();
basicRectVisual.Brush = _compositor.CreateColorBrush(Colors.Blue);
basicRectVisual.Offset = new Vector3(100, 100, 20);
basicRectVisual.Size = new Vector2(300, 300);

var basicShadow = _compositor.CreateDropShadow();
basicShadow.BlurRadius = 25f;
basicShadow.Offset = new Vector3(20, 20, 20);

basicRectVisual.Shadow = basicShadow;
```



Shaping the shadow

There are a few ways to define the shape for your DropShadow:

- **Use the default** - By default the DropShadow shape is defined by the 'Default' mode on CompositionDropShadowSourcePolicy. For SpriteVisual, the Default is Rectangular unless a mask is provided. For LayerVisual, Default is to inherit a mask using the alpha of the visual's brush.
- **Set a mask** – You may set the [Mask](#) property to define an opacity mask for the shadow.
- **Specify to use Inherited mask** – Set the [SourcePolicy](#) property to use [CompositionDropShadowSourcePolicy.InheritFromVisualContent](#) to use the mask generated from the alpha of the visual's brush.

Masking to match your content

If you want your shadow to match the Visual's content you can either use the Visual's brush for your Shadow mask property, or set the shadow to automatically inherit mask from the content. If using a LayerVisual, the shadow will inherit the mask by default.

C#

```
var imageSurface = LoadedImageSurface.StartLoadFromUri(new Uri("ms-appx:///Assets/myImage.png"));
var imageBrush = _compositor.CreateSurfaceBrush(imageSurface);

var imageSpriteVisual = _compositor.CreateSpriteVisual();
imageSpriteVisual.Size = new Vector2(400,400);
imageSpriteVisual.Offset = new Vector3(100, 500, 20);
imageSpriteVisual.Brush = imageBrush;

var shadow = _compositor.CreateDropShadow();
shadow.Mask = imageBrush;
// or use shadow.SourcePolicy =
CompositionDropShadowSourcePolicy.InheritFromVisualContent;
shadow.BlurRadius = 25f;
shadow.Offset = new Vector3(20, 20, 20);

imageSpriteVisual.Shadow = shadow;
```



Using an alternative mask

In some cases, you may want to shape the shadow such that it doesn't match your Visual's content. To achieve this effect, you'll need to explicitly set the Mask property using a brush with alpha.

In the below example, we load two surfaces - one for the Visual content and one for the Shadow mask:

C#

```
var imageSurface = LoadedImageSurface.StartLoadFromUri(new Uri("ms-appx:///Assets/myImage.png"));
var imageBrush = _compositor.CreateSurfaceBrush(imageSurface);

var circleSurface = LoadedImageSurface.StartLoadFromUri(new Uri("ms-appx:///Assets/myCircleImage.png"));
var customMask = _compositor.CreateSurfaceBrush(circleSurface);

var imageSpriteVisual = _compositor.CreateSpriteVisual();
imageSpriteVisual.Size = new Vector2(400,400);
imageSpriteVisual.Offset = new Vector3(100, 500, 20);
imageSpriteVisual.Brush = imageBrush;

var shadow = _compositor.CreateDropShadow();
```

```
shadow.Mask = customMask;
shadow.BlurRadius = 25f;
shadow.Offset = new Vector3(20, 20, 20);

imageSpriteVisual.Shadow = shadow;
```



Animating

As is standard in the Visual Layer, DropShadow properties can be animated using Composition Animations. Below, we modify the code from the sprinkles sample above to animate the blur radius for the shadow.

C#

```
ScalarKeyFrameAnimation blurAnimation =
_compositor.CreateScalarKeyFrameAnimation();
blurAnimation.InsertKeyFrame(0.0f, 25.0f);
blurAnimation.InsertKeyFrame(0.7f, 50.0f);
blurAnimation.InsertKeyFrame(1.0f, 25.0f);
blurAnimation.Duration = TimeSpan.FromSeconds(4);
blurAnimation.IterationBehavior = AnimationIterationBehavior.Forever;
shadow.StartAnimation("BlurRadius", blurAnimation);
```

Shadows in XAML

If you want to add a shadow to more complex framework elements, there are a couple ways to interop with shadows between XAML and Composition:

1. Use the `DropShadowPanel` available in the Windows Community Toolkit. See the [DropShadowPanel documentation](#) for details on how to use it.
2. Create a Visual to use as the shadow host & tie it to the XAML handout Visual.
3. Use the Composition Sample Gallery's [SamplesCommon](#) custom `CompositionShadow` control. See the example here for usage.

Performance

Although the Visual Layer has many optimizations in place to make effects efficient and usable, generating shadows can be a relatively expensive operation depending on what options you set. Below are high level 'costs' for different types of shadows. Note that although certain shadows may be expensive they may still be appropriate to use sparingly in certain scenarios.

Shadow Characteristics	Cost
Rectangular	Low
Shadow.Mask	High
CompositionDropShadowSourcePolicy.InheritFromVisualContent	High
Static Blur Radius	Low
Animating Blur Radius	High

Additional Resources

- [Composition DropShadow API](#)
- [WindowsCompositionSamples GitHub Repo](#)

Using lights in Windows UI

Article • 03/16/2023

The Windows.UI.Composition APIs enable you to create real-time animations and effects. Composition Lighting enables 3D lighting in 2D applications. In this overview, we will run through the functionality of how to setup composition lights, identify visuals to receive each light, and use effects to define materials for your content.

ⓘ Note

To read how [XamlLight](#) objects apply [CompositionLights](#) to illuminate XAML UIElements, see [XAML lighting](#).

Composition lighting lets you create interesting UI by allowing:

- Transformation of a light independent of other objects in the scene to enable immersive scenarios like music playback scenes.
- The ability to pair an object with a light so they move together independent of the rest of the scene to enable scenarios like Fluent [Reveal](#) highlight.
- Transformation of the light and entire scene as a group to create materials and depth.

Composition lighting supports three key concepts: **Light**, **Targets**, and **SceneLightingEffect**.

Light

[CompositionLight](#) allows you to create various lights and place them in coordinate space. These lights target visuals that you wish to identify as lit by the light.

Light Types

[⌵](#) Expand table

Type	Description
AmbientLight	A light source that emits non-directional light that appears reflected by everything in the scene.
DistantLight	An infinitely large distant light source that emits light in a single direction. Like the sun.

Type	Description
PointLight	A point source of light that emits light in all directions. Like a lightbulb.
SpotLight	A light source that emits inner and outer cones of light. Like a flashlight.

Targets

When lights target a Visual (add to [Targets](#) list), the Visual and all its descendants are aware of and respond to this light source. This can be something as simple as a setting a [PointLight](#) source at the root of a tree and all visuals below react to the animation of the point lights direction.

ExclusionsFromTargets gives you the ability to remove the lighting of a visual or of a subtree of visuals in a similar manner as adding targets. Children in the tree rooted by the visual that's excluded are not lit as a result.

Sample (Targets)

In the sample below, we use a [CompositionPointLight](#) to target a XAML [TextBlock](#).

CS

```
_pointLight = _compositor.CreatePointLight();
_pointLight.Color = Colors.White;
_pointLight.CoordinateSpace = text; //set up co-ordinate space for
offset
_pointLight.Targets.Add(text); //target XAML TextBlock
```

By adding animation to the offset of the point light, a shimmering effect is easily achieved.

CS

```
_pointLight.Offset = new Vector3(-(float)TextBlock.ActualWidth,
(float)TextBlock.ActualHeight / 2, (float)TextBlock.FontSize);
```

See the complete [Text Shimmer](#) sample on GitHub to learn more.

Restrictions

There are several factors to consider when determining which content will be lit by [CompositionLight](#).

[Expand table](#)

Concept	Details
Ambient Light	Adding a non-ambient light to your scene will turn off all existing light. Items not targeted by a non-ambient light will appear black. To illuminate surrounding visuals not targeted by the light in a natural way, use an ambient light in conjunction with other lights.
Number of Lights	You may use any two non-ambient composition lights in any combination to target your UI. Ambient lights are not restricted; spot, point, and distant lights are.
Lifetime	CompositionLight may experience lifetime conditions (example: the garbage collector may recycle the light object before it is used). We recommended keeping a reference to your lights by adding lights as a member to help the application manage lifetime.
Transforms	Lights must be placed in a node above UI that uses effects like perspective transforms in your visual structure to be drawn properly.
Targets and Coordinate Space	CoordinateSpace is the visual space in which all the lights properties must be set. CompositionLight.Targets must be within the CoordinateSpace tree.

Lighting Properties

Depending on the type of light used, a light can have properties for attenuation and space. Not all types of lights use all properties.

[Expand table](#)

Property	Description
Color	The Color of the light. Lighting color values are defined by D3D Diffuse, Ambient, and Specular that defines the color being emitted. Lighting uses RGBA values for lights; the alpha color component is not used.
Direction	The direction of light. The direction in which the light is pointing is specified relative to its CoordinateSpace Visual.
Coordinate Space	Every Visual has an implicit 3D coordinate space. X direction is from left to right. Y direction is from top to bottom. Z direction is a point out of the plane. The original point of this coordinate is the upper-left corner of the visual, and the unit is Device Independent Pixel (DIP). A light's offset defined in this coordinate.
Inner and Outer Cones	Spotlights emit a cone of light that has two parts: a bright inner cone and an outer cone. Composition allows you control over inner and outer cone angles

Property	Description
	and color.
Offset	Offset of the light source relative to its coordinate space Visual.

ⓘ **Note**

When multiple lights hit the same Visual, or whenever a light's color value gets large enough to exceed 1.0, the color of the light may change because of clamping of a lights color channel.

Advanced Lighting Properties

 Expand table

Property	Description
Intensity	Controls the brightness of the light.
Attenuation	Attenuation controls how a light's intensity decreases toward the maximum distance specified by the range property. Constant, Quadratic and Linear attenuation properties can be used.

Getting Started with Lighting

Follow these general steps to add lights:

- Create and place the lights: Create lights and place them in a specified coordinate space.
- Identify objects to light: Target light at relevant visuals.
- [Optional] Define how individual objects react to lights: Use SceneLightingEffect with an EffectBrush to customize light reflection for displaying the SpriteVisual. Reflection defaults support the lighting of children of a light source's CoordinateSpace. A visual painted with a SceneLightingEffect overwrites the default lighting for that visual.

SceneLightingEffect

[SceneLightingEffect](#) is used to modify the default lighting applied to the contents of a [SpriteVisual](#) targeted by a [CompositionLight](#).

[SceneLightingEffect](#) is frequently used for material creation. A [SceneLightingEffect](#) is an effect used when you want to achieve something more complex, like enabling reflective properties on an image and/or providing an illusion of depth with a normal map. A [SceneLightingEffect](#) provides the ability to customize your UI by using the lighting properties like specular and diffuse amounts. You can further customize lighting effects with the rest of the effects pipeline allowing you individually to blend and compose different lighting reactions with your content.

ⓘ Note

Scene lighting does not produce shadows; it is an effect focused on 2D rendering. It does not take into consideration 3D lighting scenarios that include real lighting models, including shadows.

[Expand table](#)

Property	Description
Normal Map	NormalMaps create an effect of a texture where a normal pointing towards the light will be brighter and a normal pointing away will be dimmer. To add a NormalMap to your targeted visual use a CompositionSurfaceBrush using LoadedImageSurface to load a NormalMap asset.
Ambient	Ambient properties are mostly used to control the overall color reflection.
Specular	Specular reflection creates highlights on objects, making them appear shiny. You can control the level of specular reflection as well as the level of shine. These properties are manipulated to create material effects like shinny metals or glossy paper.
Diffuse	Diffused reflection scatters the light in all directions.
Reflectance Model	Reflectance Model allows you to choose between Blinn Phong and Physically Based Blinn Phong. You would choose Physically Based Blinn Phong when you want to have condensed specular highlights.

Sample (SceneLightingEffect)

The sample below shows how to add a normal map to a [SceneLightingEffect](#).

```
cs
CompositionBrush CreateNormalMapBrush(ICompositionSurface normalMapImage)
{
    var colorSourceEffect = new ColorSourceEffect()
```

```

{
    Color = Colors.White
};
var sceneLightingEffect = new SceneLightingEffect()
{
    NormalMapSource = new CompositionEffectSourceParameter("NormalMap")
};

var compositeEffect = new ArithmeticCompositeEffect()
{
    Source1 = colorSourceEffect,
    Source2 = sceneLightingEffect,
};

var factory = _compositor.CreateEffectFactory(sceneLightingEffect);

var normalMapBrush = _compositor.CreateSurfaceBrush();
normalMapBrush.Surface = normalMapImage;
normalMapBrush.Stretch = CompositionStretch.Fill;

var brush = factory.CreateBrush();
brush.SetSourceParameter("NormalMap", normalMapBrush);

return brush;
}

```

Related articles

- [Creating Materials and Lights in the Visual Layer](#) 
- [Lighting Overview](#)
- [CompositionCapabilities API](#)
- [Mathematics of Lighting](#)
- [SceneLightingEffect](#)
- [WindowsCompositionSamples GitHub Repo](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Composition brushes

Article • 03/16/2023

Everything visible on your screen from a UWP application is visible because it was painted by a Brush. Brushes enable you to paint user interface (UI) objects with content ranging from simple, solid colors to images or drawings to complex effects chain. This topic introduces the concepts of painting with CompositionBrush.

Note, when working with XAML UWP app, you can choose to paint a UIElement with a [XAML Brush](#) or a [CompositionBrush](#). Typically, it is easier and advisable to choose a XAML brush if your scenario is supported by a XAML Brush. For example, animating the color of a button, changing the fill of a text or a shape with an image. On the other hand, if you are trying to do something that is not supported by a XAML brush like painting with an animated mask or an animated nine-grid stretch or an effect chain, you can use a CompositionBrush to paint a UIElement through the use of [XamlCompositionBrushBase](#).

When working with the Visual layer, a CompositionBrush must be used to paint the area of a [SpriteVisual](#).

- [Prerequisites](#)
- [Paint with CompositionBrush](#)
 - [Paint with a solid color](#)
 - [Paint with a linear gradient](#)
 - [Paint with a radial gradient](#)
 - [Paint with an image](#)
 - [Paint with a custom drawing](#)
 - [Paint with a video](#)
 - [Paint with a filter effect](#)
 - [Paint with a CompositionBrush with an opacity mask](#)
 - [Paint with a CompositionBrush using NineGrid stretch](#)
 - [Paint using Background Pixels](#)
- [Combining CompositionBrushes](#)
- [Using a XAML Brush vs. CompositionBrush](#)
- [Related Topics](#)

Prerequisites

This overview assumes that you are familiar with the structure of a basic Composition application, as described in the [Visual layer overview](#).

Paint with a CompositionBrush

A [CompositionBrush](#) "paints" an area with its output. Different brushes have different types of output. Some brushes paint an area with a solid color, others with a gradient, image, custom drawing, or effect. There are also specialized brushes that modify the behavior of other brushes. For example, opacity mask can be used to control which area is painted by a CompositionBrush, or a nine-grid can be used to control the stretch applied to a CompositionBrush when painting an area. CompositionBrush can be of one of the following types:

 [Expand table](#)

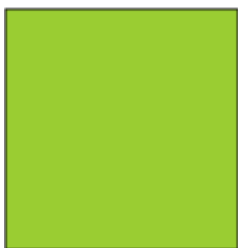
Class	Details	Introduced In
CompositionColorBrush	Paints an area with a solid color	Windows 10, version 1511 (SDK 10586)

Class	Details	Introduced In
CompositionSurfaceBrush	Paints an area with the contents of an ICompositionSurface	Windows 10, version 1511 (SDK 10586)
CompositionEffectBrush	Paints an area with the contents of a composition effect	Windows 10, version 1511 (SDK 10586)
CompositionMaskBrush	Paints a visual with a CompositionBrush with an opacity mask	Windows 10, version 1607 (SDK 14393)
CompositionNineGridBrush	Paints an area with a CompositionBrush using a NineGrid stretch	Windows 10, version 1607 (SDK 14393)
CompositionLinearGradientBrush	Paints an area with a linear gradient	Windows 10, version 1709 (SDK 16299)
CompositionRadialGradientBrush	Paints an area with a radial gradient	Windows 10, version 1903 (Insider Preview SDK)
CompositionBackdropBrush	Paints an area by sampling background pixels from either the application or pixels directly behind the application's window on desktop. Used as an input to another CompositionBrush like a CompositionEffectBrush	Windows 10, version 1607 (SDK 14393)

Paint with a solid color

A [CompositionColorBrush](#) paints an area with a solid color. There are a variety of ways to specify the color of a SolidColorBrush. For example, you can specify its alpha, red, blue, and green (ARGB) channels or use one of the predefined colors provided by the [Colors](#) class.

The following illustration and code shows a small visual tree to create a rectangle that is stroked with a black color brush and painted with a solid color brush that has the color value of 0x9ACD32.



cs

```
Compositor _compositor;
ContainerVisual _container;
SpriteVisual _colorVisual1, _colorVisual2;
CompositionColorBrush _blackBrush, _greenBrush;

_compositor = Window.Current.Compositor;
_container = _compositor.CreateContainerVisual();

_blackBrush = _compositor.CreateColorBrush(Colors.Black);
_colorVisual1 = _compositor.CreateSpriteVisual();
_colorVisual1.Brush = _blackBrush;
_colorVisual1.Size = new Vector2(156, 156);
_colorVisual1.Offset = new Vector3(0, 0, 0);
_container.Children.InsertAtBottom(_colorVisual1);

_greenBrush = _compositor.CreateColorBrush(Color.FromArgb(0xff, 0x9A, 0xCD, 0x32));
_colorVisual2 = _compositor.CreateSpriteVisual();
```

```
_colorVisual2.Brush = _greenBrush;  
_colorVisual2.Size = new Vector2(150, 150);  
_colorVisual2.Offset = new Vector3(3, 3, 0);  
_container.Children.InsertAtBottom(_colorVisual2);
```

Paint with a linear gradient

A [CompositionLinearGradientBrush](#) paints an area with a linear gradient. A linear gradient blends two or more colors across a line, the gradient axis. You use GradientStop objects to specify the colors in the gradient and their positions.

The following illustration and code shows a SpriteVisual painted with a LinearGradientBrush with 2 stops using a red and yellow color.



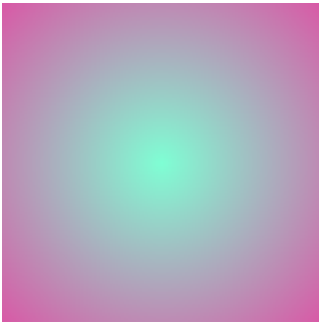
CS

```
Compositor _compositor;  
SpriteVisual _gradientVisual;  
CompositionLinearGradientBrush _redyellowBrush;  
  
_compositor = Window.Current.Compositor;  
  
_redyellowBrush = _compositor.CreateLinearGradientBrush();  
_redyellowBrush.ColorStops.Add(_compositor.CreateColorGradientStop(0, Colors.Red));  
_redyellowBrush.ColorStops.Add(_compositor.CreateColorGradientStop(1, Colors.Yellow));  
_gradientVisual = _compositor.CreateSpriteVisual();  
_gradientVisual.Brush = _redyellowBrush;  
_gradientVisual.Size = new Vector2(156, 156);
```

Paint with a radial gradient

A [CompositionRadialGradientBrush](#) paints an area with a radial gradient. A radial gradient blends two or more colors with the gradient starting from the center of the ellipse and ending at the ellipse's radius. GradientStop objects are used to define the colors and their location in the gradient.

The following illustration and code shows a SpriteVisual painted with a RadialGradientBrush with 2 GradientStops.



cs

```
Compositor _compositor;
SpriteVisual _gradientVisual;
CompositionRadialGradientBrush RGBBrush;

_compositor = Window.Current.Compositor;

RGBBrush = _compositor.CreateRadialGradientBrush();
RGBBrush.ColorStops.Add(_compositor.CreateColorGradientStop(0, Colors.Aquamarine));
RGBBrush.ColorStops.Add(_compositor.CreateColorGradientStop(1, Colors.DeepPink));
_gradientVisual = _compositor.CreateSpriteVisual();
_gradientVisual.Brush = RGBBrush;
_gradientVisual.Size = new Vector2(200, 200);
```

Paint with an image

A [CompositionSurfaceBrush](#) paints an area with pixels rendered onto an [ICompositionSurface](#). For example, a [CompositionSurfaceBrush](#) can be used to paint an area with an image rendered onto an [ICompositionSurface](#) surface using [LoadedImageSurface](#) API.

The following illustration and code shows a [SpriteVisual](#) painted with a bitmap of a licorice rendered onto an [ICompositionSurface](#) using [LoadedImageSurface](#). The properties of [CompositionSurfaceBrush](#) can be used to stretch and align the bitmap within the bounds of the visual.



cs

```
Compositor _compositor;
SpriteVisual _imageVisual;
CompositionSurfaceBrush _imageBrush;

_compositor = Window.Current.Compositor;

_imageBrush = _compositor.CreateSurfaceBrush();

// The loadedSurface has a size of 0x0 till the image has been downloaded, decoded and loaded to the
// surface. We can assign the surface to the CompositionSurfaceBrush and it will show up once the image
// is loaded to the surface.
LoadedImageSurface _loadedSurface = LoadedImageSurface.StartLoadFromUri(new Uri("ms-
appx:///Assets/licorice.jpg"));
_imageBrush.Surface = _loadedSurface;

_imageVisual = _compositor.CreateSpriteVisual();
```

```
_imageVisual.Brush = _imageBrush;  
_imageVisual.Size = new Vector2(156, 156);
```

Paint with a custom drawing

A [CompositionSurfaceBrush](#) can also be used to paint an area with pixels from an [ICompositionSurface](#) rendered using [Win2D](#) (or D2D).

The following code shows a [SpriteVisual](#) painted with a text run rendered onto an [ICompositionSurface](#) using [Win2D](#). Note, in order to use [Win2D](#) you need to include the [Win2D NuGet](#) package into your project.

```
cs  
  
Compositor _compositor;  
CanvasDevice _device;  
CompositionGraphicsDevice _compositionGraphicsDevice;  
SpriteVisual _drawingVisual;  
CompositionSurfaceBrush _drawingBrush;  
  
_device = CanvasDevice.GetSharedDevice();  
_compositionGraphicsDevice = CanvasComposition.CreateCompositionGraphicsDevice(_compositor,  
_device);  
  
_drawingBrush = _compositor.CreateSurfaceBrush();  
CompositionDrawingSurface _drawingSurface = _compositionGraphicsDevice.CreateDrawingSurface(new  
Size(256, 256), DirectXPixelFormat.B8G8R8A8UIntNormalized, DirectXAlphaMode.Premultiplied);  
  
using (var ds = CanvasComposition.CreateDrawingSession(_drawingSurface))  
{  
    ds.Clear(Colors.Transparent);  
    var rect = new Rect(new Point(2, 2), (_drawingSurface.Size.ToVector2() - new Vector2(4,  
4)).ToSize());  
    ds.FillRoundedRectangle(rect, 15, 15, Colors.LightBlue);  
    ds.DrawRoundedRectangle(rect, 15, 15, Colors.Gray, 2);  
    ds.DrawText("This is a composition drawing surface", rect, Colors.Black, new CanvasTextFormat()  
    {  
        FontFamily = "Comic Sans MS",  
        FontSize = 32,  
        WordWrapping = CanvasWordWrapping.WholeWord,  
        VerticalAlignment = CanvasVerticalAlignment.Center,  
        HorizontalAlignment = CanvasHorizontalAlignment.Center  
    }  
);  
  
_drawingBrush.Surface = _drawingSurface;  
  
_drawingVisual = _compositor.CreateSpriteVisual();  
_drawingVisual.Brush = _drawingBrush;  
_drawingVisual.Size = new Vector2(156, 156);
```

Similarly, the [CompositionSurfaceBrush](#) can also be used to paint a [SpriteVisual](#) with a [SwapChain](#) using [Win2D](#) interop. [This sample](#) provides an example of how to use [Win2D](#) to paint a [SpriteVisual](#) with a [swapchain](#).

Paint with a video

A [CompositionSurfaceBrush](#) can also be used to paint an area with pixels from an [ICompositionSurface](#) rendered using a video loaded through the [MediaPlayer](#) class.

The following code shows a [SpriteVisual](#) painted with a video loaded onto an [ICompositionSurface](#).

```
cs
```

```

Compositor _compositor;
SpriteVisual _videoVisual;
CompositionSurfaceBrush _videoBrush;

// MediaPlayer set up with a create from URI

_mediaPlayer = new MediaPlayer();

// Get a source from a URI. This could also be from a file via a picker or a stream
var source = MediaSource.CreateFromUri(new Uri("https://go.microsoft.com/fwlink/?
LinkID=809007&clcid=0x409"));
var item = new MediaPlaybackItem(source);
_mediaPlayer.Source = item;
_mediaPlayer.IsLoopingEnabled = true;

// Get the surface from MediaPlayer and put it on a brush
_videoSurface = _mediaPlayer.GetSurface(_compositor);
_videoBrush = _compositor.CreateSurfaceBrush(_videoSurface.CompositionSurface);

_videoVisual = _compositor.CreateSpriteVisual();
_videoVisual.Brush = _videoBrush;
_videoVisual.Size = new Vector2(156, 156);

```

Paint with a filter effect

A [CompositionEffectBrush](#) paints an area with output of a CompositionEffect. Effects in the Visual Layer may be thought of as animatable filter effects applied to a collection of source content such as colors, gradients, images, videos, swapchains, regions of your UI, or trees of Visuals. The source content is typically specified using another CompositionBrush.

The following illustration and code shows a SpriteVisual painted with an image of a cat that has desaturation filter effect applied.



CS

```

Compositor _compositor;
SpriteVisual _effectVisual;
CompositionEffectBrush _effectBrush;

_compositor = Window.Current.Compositor;

var graphicsEffect = new SaturationEffect {
    Saturation = 0.0f,
    Source = new CompositionEffectSourceParameter("mySource")
};

```

```

var effectFactory = _compositor.CreateEffectFactory(graphicsEffect);
_effectBrush = effectFactory.CreateBrush();

CompositionSurfaceBrush surfaceBrush = _compositor.CreateSurfaceBrush();
LoadedImageSurface loadedSurface = LoadedImageSurface.StartLoadFromUri(new Uri("ms-appx:///Assets/cat.jpg"));
SurfaceBrush.surface = loadedSurface;

_effectBrush.SetSourceParameter("mySource", surfaceBrush);

_effectVisual = _compositor.CreateSpriteVisual();
_effectVisual.Brush = _effectBrush;
_effectVisual.Size = new Vector2(156, 156);

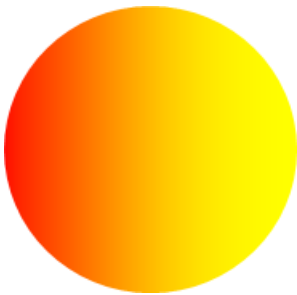
```

For more information on creating an Effect using CompositionBrushes see [Effects in Visual layer](#)

Paint with a CompositionBrush with opacity mask applied

A [CompositionMaskBrush](#) paints an area with a CompositionBrush with an opacity mask applied to it. The source of the opacity mask can be any CompositionBrush of type CompositionColorBrush, CompositionLinearGradientBrush, CompositionSurfaceBrush, CompositionEffectBrush, or CompositionNineGridBrush. The opacity mask must be specified as a CompositionSurfaceBrush.

The following illustration and code shows a SpriteVisual painted with a CompositionMaskBrush. The source of the mask is a CompositionLinearGradientBrush which is masked to look like a circle using an image of circle as a mask.



cs

```

Compositor _compositor;
SpriteVisual _maskVisual;
CompositionMaskBrush _maskBrush;

_compositor = Window.Current.Compositor;

_maskBrush = _compositor.CreateMaskBrush();

CompositionLinearGradientBrush _sourceGradient = _compositor.CreateLinearGradientBrush();
_sourceGradient.ColorStops.Add(_compositor.CreateColorGradientStop(0, Colors.Red));
_sourceGradient.ColorStops.Add(_compositor.CreateColorGradientStop(1, Colors.Yellow));
_maskBrush.Source = _sourceGradient;

LoadedImageSurface loadedSurface = LoadedImageSurface.StartLoadFromUri(new Uri("ms-appx:///Assets/circle.png"), new Size(156.0, 156.0));
_maskBrush.Mask = _compositor.CreateSurfaceBrush(loadedSurface);

_maskVisual = _compositor.CreateSpriteVisual();
_maskVisual.Brush = _maskBrush;
_maskVisual.Size = new Vector2(156, 156);

```

Paint with a CompositionBrush using NineGrid stretch

A [CompositionNineGridBrush](#) paints an area with a CompositionBrush that is stretched using the nine-grid metaphor. The nine-grid metaphor enables you to stretch edges and corners of a CompositionBrush differently than its center. The source of the nine-grid stretch can be any CompositionBrush of type CompositionColorBrush, CompositionSurfaceBrush, or CompositionEffectBrush.

The following code shows a SpriteVisual painted with a CompositionNineGridBrush. The source of the mask is a CompositionSurfaceBrush which is stretched using a Nine-Grid.

```
cs

Compositor _compositor;
SpriteVisual _nineGridVisual;
CompositionNineGridBrush _nineGridBrush;

_compositor = Window.Current.Compositor;

_nineGridBrush = _compositor.CreateNineGridBrush();

// nineGridImage.png is 50x50 pixels; nine-grid insets, as measured relative to the actual size of
// the image, are: left = 1, top = 5, right = 10, bottom = 20 (in pixels)

LoadedImageSurface _imageSurface = LoadedImageSurface.StartLoadFromUri(new Uri("ms-
appx:///Assets/nineGridImage.png"));
_nineGridBrush.Source = _compositor.CreateSurfaceBrush(_imageSurface);

// set Nine-Grid Insets

_nineGridBrush.SetInsets(1, 5, 10, 20);

// set appropriate Stretch on SurfaceBrush for Center of Nine-Grid

sourceBrush.Stretch = CompositionStretch.Fill;

_nineGridVisual = _compositor.CreateSpriteVisual();
_nineGridVisual.Brush = _nineGridBrush;
_nineGridVisual.Size = new Vector2(100, 75);
```

Paint using Background Pixels

A [CompositionBackdropBrush](#) paints an area with the content behind the area. A CompositionBackdropBrush is never used on its own, but instead is used as an input to another CompositionBrush like an EffectBrush. For example, by using a CompositionBackdropBrush as an input to a Blur effect, you can achieve a frosted glass effect.

The following code shows a small visual tree to create an image using CompositionSurfaceBrush and a frosted glass overlay above the image. The frosted glass overlay is created by placing a SpriteVisual filled with an EffectBrush above the image. The EffectBrush uses a CompositionBackdropBrush as an input to the blur effect.

```
cs

Compositor _compositor;
SpriteVisual _containerVisual;
SpriteVisual _imageVisual;
SpriteVisual _backdropVisual;

_compositor = Window.Current.Compositor;

// Create container visual to host the visual tree
_containerVisual = _compositor.CreateContainerVisual();

// Create _imageVisual and add it to the bottom of the container visual.
// Paint the visual with an image.
```

```

CompositionSurfaceBrush _licoriceBrush = _compositor.CreateSurfaceBrush();

LoadedImageSurface loadedSurface =
    LoadedImageSurface.StartLoadFromUri(new Uri("ms-appx:///Assets/licorice.jpg"));
_licoriceBrush.Surface = loadedSurface;

_imageVisual = _compositor.CreateSpriteVisual();
_imageVisual.Brush = _licoriceBrush;
_imageVisual.Size = new Vector2(156, 156);
_imageVisual.Offset = new Vector3(0, 0, 0);
_containerVisual.Children.InsertAtBottom(_imageVisual)

// Create a SpriteVisual and add it to the top of the containerVisual.
// Paint the visual with an EffectBrush that applies blur to the content
// underneath the Visual to create a frosted glass effect.

GaussianBlurEffect blurEffect = new GaussianBlurEffect(){
    Name = "Blur",
    BlurAmount = 1.0f,
    BorderMode = EffectBorderMode.Hard,
    Source = new CompositionEffectSourceParameter("source");
};

CompositionEffectFactory blurEffectFactory = _compositor.CreateEffectFactory(blurEffect);
CompositionEffectBrush _backdropBrush = blurEffectFactory.CreateBrush();

// Create a BackdropBrush and bind it to the EffectSourceParameter source

_backdropBrush.SetSourceParameter("source", _compositor.CreateBackdropBrush());
_backdropVisual = _compositor.CreateSpriteVisual();
_backdropVisual.Brush = _licoriceBrush;
_backdropVisual.Size = new Vector2(78, 78);
_backdropVisual.Offset = new Vector3(39, 39, 0);
_containerVisual.Children.InsertAtTop(_backdropVisual);

```

Combining CompositionBrushes

A number of CompositionBrushes use other CompositionBrushes as inputs. For example, using the SetSourceParameter method can be used to set another CompositionBrush as an input to a CompositionEffectBrush. The table below outlines the supported combinations of CompositionBrushes. Note, that using an unsupported combination will throw an exception.

[Expand table](#)

Brush	EffectBrush.SetSourceParameter()	MaskBrush.Mask	MaskBrush.Source	NineGridBrush.Source
CompositionColorBrush	YES	YES	YES	YES
CompositionLinear GradientBrush	YES	YES	YES	NO
CompositionSurfaceBrush	YES	YES	YES	YES
CompositionEffectBrush	NO	NO	YES	NO
CompositionMaskBrush	NO	NO	NO	NO
CompositionNineGridBrush	YES	YES	YES	NO
CompositionBackdropBrush	YES	NO	NO	NO

Using a XAML Brush vs. CompositionBrush

The following table provides a list of scenarios and whether XAML or Composition brush use is prescribed when painting a UIElement or a SpriteVisual in your application.

 Note

If a CompositionBrush is suggested for a XAML UIElement, it is assumed that the CompositionBrush is packaged using a XamlCompositionBrushBase.

 Expand table

Scenario	XAML UIElement	Composition SpriteVisual
Paint an area with solid color	SolidColorBrush	CompositionColorBrush
Paint an area with animated color	SolidColorBrush	CompositionColorBrush
Paint an area with a static gradient	LinearGradientBrush	CompositionLinearGradientBrush
Paint an area with animated gradient stops	CompositionLinearGradientBrush	CompositionLinearGradientBrush
Paint an area with an image	ImageBrush	CompositionSurfaceBrush
Paint an area with a webpage	WebViewBrush	N/A
Paint an area with an image using NineGrid stretch	Image Control	CompositionNineGridBrush
Paint an area with animated NineGrid stretch	CompositionNineGridBrush	CompositionNineGridBrush
Paint an area with a swapchain	SwapChainPanel	CompositionSurfaceBrush w/ swapchain interop
Paint an area with a video	MediaElement	CompositionSurfaceBrush w/ media interop
Paint an area with custom 2D drawing	CanvasControl  from Win2D	CompositionSurfaceBrush w/ Win2D interop
Paint an area with non-animated mask	Use XAML shapes to define a mask	CompositionMaskBrush
Paint an area with an animated mask	CompositionMaskBrush	CompositionMaskBrush
Paint an area with an animated filter effect	CompositionEffectBrush	CompositionEffectBrush
Paint an area with an effect applied to background pixels	CompositionBackdropBrush	CompositionBackdropBrush

Related Topics

[Composition native DirectX and Direct2D interop with BeginDraw and EndDraw](#)

[XAML brush interop with XamlCompositionBrushBase](#)

Feedback

Was this page helpful?



[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Composition native interoperation with DirectX and Direct2D

Article • 10/20/2022

The Windows.UI.Composition API provides the [ICompositorInterop](#), [ICompositionDrawingSurfaceInterop](#), and [ICompositionGraphicsDeviceInterop](#) native interoperation interfaces allowing content to be moved directly into the compositor.

Native interoperation is structured around surface objects that are backed by DirectX textures. The surfaces are created from a factory object called [CompositionGraphicsDevice](#). This object is backed by an underlying Direct2D or Direct3D device object, which it uses to allocate video memory for surfaces. The composition API never creates the underlying DirectX device. It is the responsibility of the application to create one and pass it to the [CompositionGraphicsDevice](#) object. An application may create more than one [CompositionGraphicsDevice](#) object at a time, and it may use the same DirectX device as the rendering device for multiple [CompositionGraphicsDevice](#) objects.

Creating a surface

Each [CompositionGraphicsDevice](#) serves as a surface factory. Each surface is created with an initial size (which may be 0,0), but no valid pixels. A surface in its initial state may be immediately consumed in a visual tree, for example, via a [CompositionSurfaceBrush](#) and a [SpriteVisual](#), but in its initial state the surface has no effect on screen output. It is, for all purposes, entirely transparent, even if the specified alpha mode is "opaque".

Occasionally, DirectX devices may be rendered unusable. This may happen, amongst other reasons, if the application passes invalid arguments to certain DirectX APIs, or if the graphics adapter is reset by the system, or if the driver is updated. Direct3D has an API that an application may use to discover, asynchronously, if the device is lost for any reason. When a DirectX device is lost, the application must discard it, create a new one, and pass it to any [CompositionGraphicsDevice](#) objects previously associated with the bad DirectX device.

Loading pixels into a surface

To load pixels into the surface, the application must call the [BeginDraw](#) method, which returns a DirectX interface representing a texture or Direct2D context, depending on what the application requests. The application must then render or upload pixels into

that texture. When the application is done, it must call the **EndDraw** method. Only at that point are the new pixels available for composition, but they still don't show up on screen until the next time all changes to the visual tree are committed. If the visual tree is committed before **EndDraw** is called, then the update that is in progress is not visible on screen and the surface continues to display the contents it had prior to **BeginDraw**. When **EndDraw** is called, the texture or Direct2D context pointer returned by **BeginDraw** is invalidated. An application should never cache that pointer beyond the **EndDraw** call.

The application may only call **BeginDraw** on one surface at a time, for any given **CompositionGraphicsDevice**. After calling **BeginDraw**, the application must call **EndDraw** on that surface before calling **BeginDraw** on another. As the API is agile, the application is responsible for synchronizing these calls if it wishes to perform rendering from multiple worker threads. If an application wants to interrupt rendering one surface and switch to another temporarily, the application may use the **SuspendDraw** method. This allows another **BeginDraw** to succeed, but does not make the first surface update available for on-screen composition. This allows the application to perform multiple updates in a transactional manner. Once a surface is suspended, the application may continue the update by calling the **ResumeDraw** method, or it may declare that the update is done by calling **EndDraw**. This means only one surface can be actively updated at a time for any given **CompositionGraphicsDevice**. Each graphics device keeps this state independently of the others, so an application may render to two surfaces simultaneously if they belong to different graphics devices. However, this precludes the video memory for those two surfaces from being pooled together and, as such, is less memory efficient.

The **BeginDraw**, **SuspendDraw**, **ResumeDraw** and **EndDraw** methods return failures if the application performs an incorrect operation (such as passing invalid arguments, or calling **BeginDraw** on a surface before calling **EndDraw** on another). These types of failures represent application bugs and, as such, the expectation is that they are handled with a fail fast. **BeginDraw** may also return a failure if the underlying DirectX device is lost. This failure is not fatal as the application can recreate its DirectX device and try again. As such, the application is expected to handle device loss by simply skipping rendering. If **BeginDraw** fails for any reason, the application should also not call **EndDraw**, as the begin never succeeded in the first place.

Scrolling

For performance reasons, when an application calls **BeginDraw** the contents of the returned texture are not guaranteed to be the previous contents of the surface. The application must assume that the contents are random and, as such, the application must ensure that all pixels are touched, either by clearing the surface before rendering

or by drawing enough opaque contents to cover the entire updated rectangle. This, combined with the fact that the texture pointer is only valid between **BeginDraw** and **EndDraw** calls, makes it impossible for the application to copy previous contents out of the surface. For this reason, we offer a **Scroll** method, which allows the application to perform a same-surface pixel copy.

C++/WinRT usage example

The following code example illustrates an interoperation scenario. The example combines types from the Windows Runtime-based surface area of Windows Composition, together with types from the interop headers, and code that renders text using the COM-based DirectWrite and Direct2D APIs. The example uses **BeginDraw** and **EndDraw** to make it seamless to interoperate between these technologies. The example uses DirectWrite to lay out the text, and then it uses Direct2D to render it. The composition graphics device accepts the Direct2D device directly at initialization time. This allows **BeginDraw** to return an **ID2D1DeviceContext** interface pointer, which is considerably more efficient than having the application create a Direct2D context to wrap a returned **ID3D11Texture2D** interface at each drawing operation.

To try out the C++/WinRT code example below, first create a new **Core App (C++/WinRT)** project in Visual Studio (for requirements, see [Visual Studio support for C++/WinRT](#)). Replace the contents of your `pch.h` and `App.cpp` source code files with the code listings below, then build and run. The application renders the string "Hello, World!" in black text on a transparent background.

C++/WinRT

```
// pch.h
#pragma once
#include <windows.h>
#include <D2d1_1.h>
#include <D3d11_4.h>
#include <Dwrite.h>
#include <Windows.Graphics.DirectX.Direct3D11.interop.h>
#include <Windows.ui.composition.interop.h>
#include <unknwn.h>

#include <winrt/Windows.ApplicationModel.Core.h>
#include <winrt/Windows.Foundation.h>
#include <winrt/Windows.Graphics.DirectX.h>
#include <winrt/Windows.Graphics.DirectX.Direct3D11.h>
#include <winrt/Windows.UI.Composition.h>
#include <winrt/Windows.UI.Core.h>
#include <winrt/Windows.UI.Input.h>
```

```

// App.cpp
//*****
//
// Copyright (c) Microsoft. All rights reserved.
// This code is licensed under the MIT License (MIT).
// THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
// OR IMPLIED,
// INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
// FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
// IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
// CLAIM,
// DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
// TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH
// THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
//
//*****

#include "pch.h"

using namespace winrt;
using namespace winrt::Windows::ApplicationModel::Core;
using namespace winrt::Windows::Foundation;
using namespace winrt::Windows::Foundation::Numerics;
using namespace winrt::Windows::Graphics::DirectX;
using namespace winrt::Windows::Graphics::DirectX::Direct3D11;
using namespace winrt::Windows::UI;
using namespace winrt::Windows::UI::Composition;
using namespace winrt::Windows::UI::Core;

namespace abi
{
    using namespace ABI::Windows::Foundation;
    using namespace ABI::Windows::Graphics::DirectX;
    using namespace ABI::Windows::UI::Composition;
}

// An app-provided helper to render lines of text.
struct SampleText
{
    SampleText(winrt::com_ptr<::IDWriteTextLayout> const& text,
        CompositionGraphicsDevice const& compositionGraphicsDevice) :
        m_text(text),
        m_compositionGraphicsDevice(compositionGraphicsDevice)
    {
        // Create the surface just big enough to hold the formatted text
        block.

        DWRITE_TEXT_METRICS metrics;
        winrt::check_hresult(m_text->GetMetrics(&metrics));
        winrt::Windows::Foundation::Size surfaceSize{ metrics.width,
        metrics.height };

        CompositionDrawingSurface drawingSurface{

```

```

m_compositionGraphicsDevice.CreateDrawingSurface(
    surfaceSize,
    DirectXPixelFormat::B8G8R8A8UIntNormalized,
    DirectXAlphaMode::Premultiplied) };

    // Cache the interop pointer, since that's what we always use.
    m_drawingSurfaceInterop =
drawingSurface.as<abi::ICompositionDrawingSurfaceInterop>();

    // Draw the text
    DrawText();

    // If the rendering device is lost, the application will recreate
and replace it. We then
    // own redrawing our pixels.
    m_deviceReplacedEventToken =
m_compositionGraphicsDevice.RenderingDeviceReplaced(
    [this](CompositionGraphicsDevice const&,
RenderingDeviceReplacedEventArgs const&)
    {
        // Draw the text again.
        DrawText();
        return S_OK;
    });
}

~SampleText()
{

m_compositionGraphicsDevice.RenderingDeviceReplaced(m_deviceReplacedEventTok
en);
}

    // Return the underlying surface to the caller.
    auto Surface()
    {
        // To the caller, the fact that we have a drawing surface is an
implementation detail.
        // Return the base interface instead.
        return m_drawingSurfaceInterop.as<ICompositionSurface>();
    }

private:
    // The text to draw.
    winrt::com_ptr<::IDWriteTextLayout> m_text;

    // The composition surface that we use in the visual tree.
    winrt::com_ptr<abi::ICompositionDrawingSurfaceInterop>
m_drawingSurfaceInterop;

    // The device that owns the surface.
    CompositionGraphicsDevice m_compositionGraphicsDevice{ nullptr };
    //winrt::com_ptr<abi::ICompositionGraphicsDevice>
m_compositionGraphicsDevice2;

```